# DATA STRUCTURES & ALGORITHM

— By C++

## BEGINNER → ADVANCE GUIDE

ABHISHEK RATHOR

INSTAGRAM — @Syntax Error

# INDEX

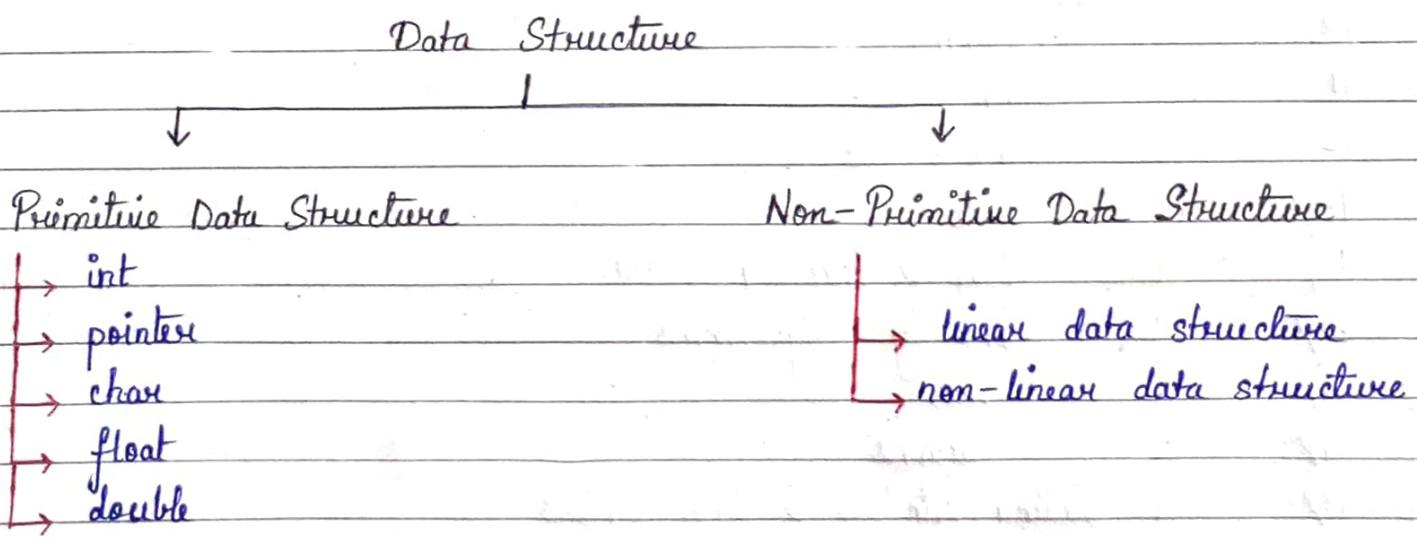

| Sl. No. | Topic | Pg No |
|---|---|---|
| 1) | Data Structure Introduction | |
| 2) | Classification of data structure | |
| 3) | Introduction to algorithm | |
| 4) | Asymptotic Notation | |
| 5) | Basics of C++ | |
| 6) | Arrays | |
| 7) | Strings | |
| 8) | Searching & Binary search | |
| 9) | Recursion & Backtracking | |
| 10) | Hashing | |
| 11) | Stack & Queue | |
| 12) | Linked List | |
| 13) | Tree | |
| 14) | Heap & Priority Queue | |
| 15) | Greedy Algorithms | |
| 16) | Tries | |
| 17) | Graphs | |
| 18) | Sliding Window & Two Pointers | |
| 19) | Bit Manipulation | |
| 20) | Segment Tree & Fenwick Tree | |
| 21) | Lazy Propagation | |
| 22) | Interview & Contest Specific Topics | |

# What is Data Structure?

→ Data Structure is a way to store and organise data so that it can be used effeciently.

→ As per name indicates itself that organizing that data in memory.

→ The data structure is not any programming language like C, C++, Java etc. It is a set of algorithms that can be used in any programming language to structure data in memory.




## Linear Data Structure

→ The arrangement of Data in the sequential manner is known as linear Data Structure.

→ The data structure used for this purpose are Arrays, Linked-List, Stacks and queues.

→ In this Data structure, one element is connected to only one another element in a linear form.

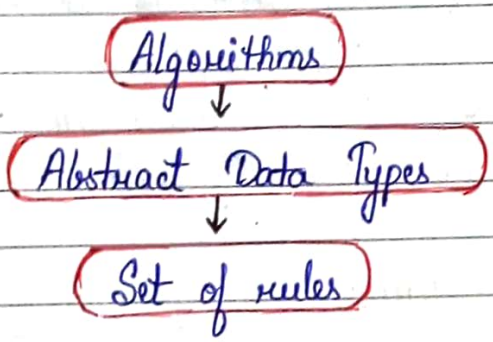
## Non-Linear Data Structure

→ When one element is connected to the 'n' number of elements known as non-linear structure.

Examples – Trees & Graphs

→ In this case, elements are arranged in a random manner.

## Algorithms and Abstract Data Types




Why ??

To structure the data in memory, 'n' number of algorithms are proposed and all these algorithms are known as Abstract Data Types (ADT).

An Abstract Data Type tells what is to be done and data structure tells how is to done ?

ADT gives us the blueprint while data structure provides the implementation part.

## What is Data?

→ Data can be defined as the elementary value / collection of values.

→ For example – Student's name and its id are the data about student.

What is Record ?

→ Record can be defined as collection of various data items.

→ Examples – Student entity
↳ name, address, course and marks can be grouped together to form record.

What is File ?

→ File is a collection of various records of one type of entity.

→ Example – If there are 60 employees in class then there will be 20 records in related file where record contains info of employees.

What is Attribute and Entity ?

An entity represents class of certain objects, it contains various attribute each attribute represents particular property of that entity.

# What is need of Data Structure ?

As applications are getting complexed and amount of data is increasing day by day, there may arise following problems –

→ Processor Speed – As data is growing day by day to the billions of files per entity, processor may fail to deal with that amount of data.

→ Data Structure – Consider an inventory size of 100 items in store, if our application needs to search for a particular item, it needs traverse 100 items every time, results in slowing down process.

→ Multiple Requests – If thousands of users are searching data simultaneously on a web server, then there are chances that to be failed to search during that process.

To solve these problems, data structures are used Data is organized to form a data structure in such a way that all items are not required to be searched and require data can be searched instantly.

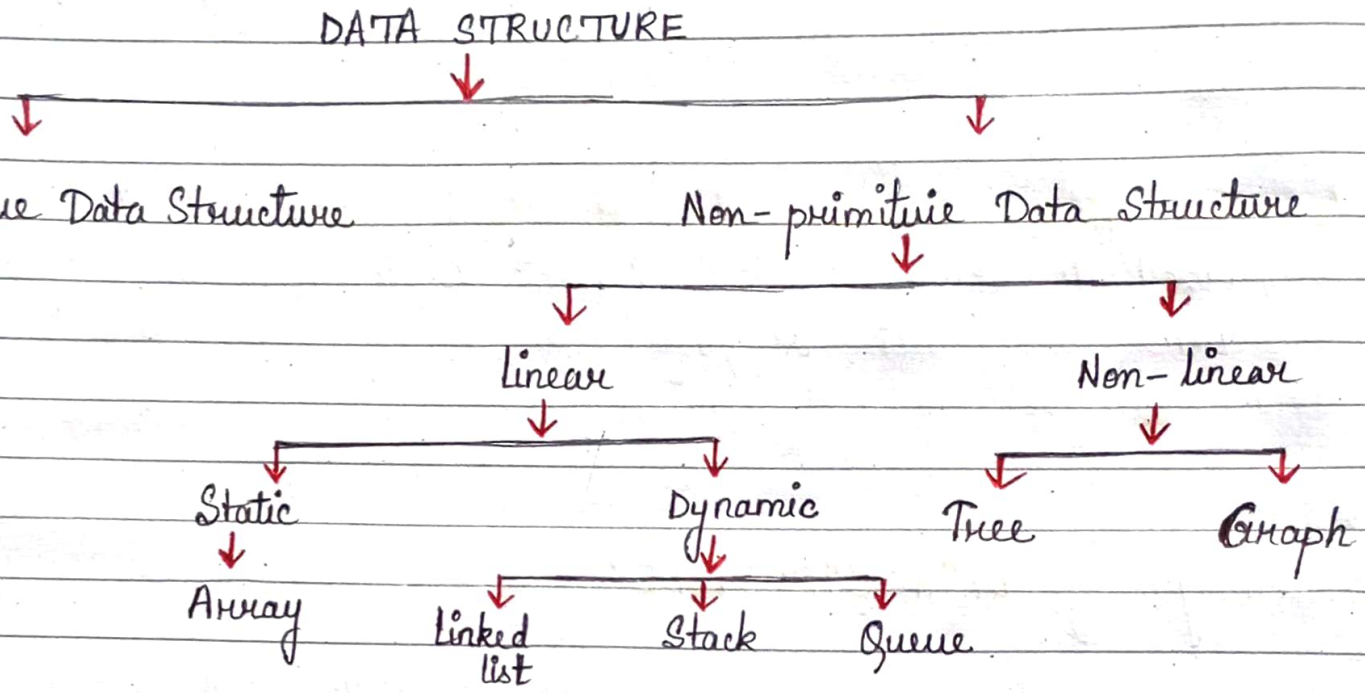
Advantages of DATA STRUCTURE:

1. Efficiency – If the choice of a data structure for implementing a particular ADT is proper, it makes program very efficient in terms of time and space.

2. Reusability – The data structure provides reusability means that multiple client programs can use the data structure.

3. Abstraction – The data structure specified by the ADT also provides level of abstraction. The client cannot see internal working of data structure, so it does not have to worry about implementation

## DATA STRUCTURE Classification —




## Operations on DATA STRUCTURE

1. Traversing – Every data structure contains a set of data elements. Traversing data structure means visiting each elements of data structure in order to perform some specific operation like searching or sorting.

   Example – If we need to calculate average of marks obtained by a student in n different subject, we need to traverse compute array of marks and calculate total sum, then we will divide that sum by no. of subjects i.e. to find average.

2. Insertion – Insertion can be defined as the process of adding the elements to the data structure at any location.

If the size of data structure is n then we can only insert n-1 data elements to it.

3. Deletion - The process of removing an element from the data structure is called deletion. We can delete an element from data structure at any random location. If we try to delete an element from an empty data structure, then underflow occurs.

4. Searching - The process of finding the location of an element within data structure is called searching. There are two algorithms to perform searching, linear search & binary search.

5. Sorting - The process of arranging the data structure in a specific order is called as sorting. There are many algorithms that can be used to perform sorting. For example - Insertion sort, Selection sort, Bubble sort etc.

6. Merging - When two lists List A and list B of size m & n respectively of similar types of elements clubbed or joined to produce third list, List C of size (m+n), then this process is called merging.

# Data Structure and Algorithm

## # What is Algorithm?

→ An algorithm is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer.

→ It is not complete program or code, it is just a solution (logic) of a problem, which can be represented either as an informal description using a flowchart or pseudocode.

## Characteristics of an Algorithm

- **Input :** An algorithm has some input values. We can pass one or more input values to an algorithm.
- **Output :** We will get one or more outputs at the end of an algorithm.
- **Unambiguity :** An algorithm should be unambiguous, which means that each step in the algorithm should be clear and simple.
- **Finiteness :** An algorithm should have a finite (limited) number of steps of instructions.

- Effectiveness – An algorithm should be practical and each step of the algorithm should take a finite amount of time and be feasible to perform.

Approaches in Algorithm:

1) Brute force Algorithm – The general logic structure is applied to design an algorithm. It is also known as exhaustive search algorithm that searches all possible to provide required solution.

Such algorithms have two types:

1) Optimizing: Finding all solutions of a problem and then take out the best solution is known that it will terminate if the best solution is known

2) Sacrificing: As soon as the best solution is found, then it will stop.

→ Divide and conquer: This breaks down the algorithm to solve the problem in different methods. It allows you to break down problem into different methods, and valid output is produced for the valid input. This valid output is passed to some other function.

→ Greedy Algorithm: It is an algorithm paradigm that makes an optimal choice on each iteration with the hope of getting best solution. It is easy to implement and has faster

execution time. But there are very rare cases in which it provides the optimal solution.

The major categories of algorithms are given below –

- Sort : Algorithm developed for sorting the items in a certain order.
- Search : Algorithm developed for searching the items inside a data structure.
- Delete : Algorithm developed for deleting the existing element from the data structure.
- Insert : Algorithm developed for insert an item inside a data structure.
- Update : Algorithm developed for updating the existing element inside a data structure.
- Algorithm Analysis : The algorithm can be analyzed in two levels i.e first is before creating the algorithm and second is after creating the algorithm.

There are two analysis of an algorithm

- Priori Analysis –
  Here, priori analysis is the theoretical analysis of an algorithm, which is done before implementing the algorithm.

• Posterior Analysis —
Here, posterior analysis is a practical analysis of an algorithm. The practical analysis is achieved by implementing the algorithm using any programming language.

→ Algorithm Complexity :
The performance of the algorithm can be measured in two factors —

→ Time Complexity : The time complexity of an algorithm is the amount of the time required to complete the execution. The time complexity of an algorithm is denoted by the big O notation.

Here, big O notation is the asymptotic notation to represent time complexity. The time complexity is mainly calculated by counting the number of steps to finish execution.

```
sum = 0;      // suppose we have to calculate the sum of n numbers.
for i=1 to n.
sum = sum + i;  // when the loop ends then sum holds the sum
return sum;                    of n numbers.
```

In the above code, the time complexity of the loop statement will be atleast n, and if value of n increases, then time complexity also increases.

We generally consider the worst-time complexity as it is maximum time taken for any given input size.

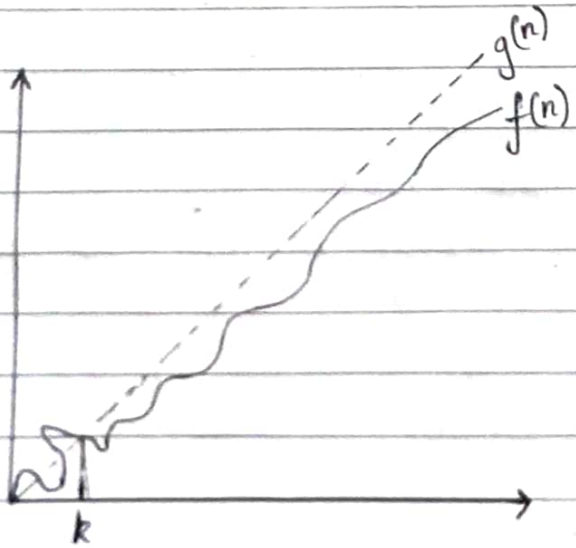
→ Worst case : Maximum time taken by an algorithm for any input.

→ Average case : Expected time over all inputs.
→ Best case : Minimum time the algorithm takes on any input.

Asymptotic analysis uses notations to describe the running time in a way that ignores constant factors and lower-order terms

## 1. Big O Notation (O)

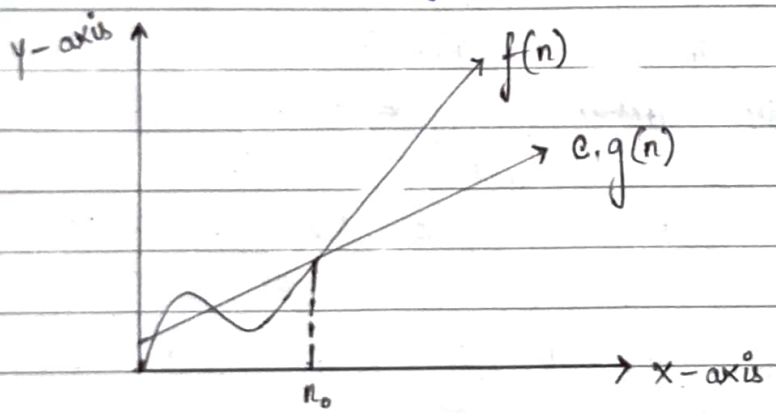
- Describes the upper bound of the running time of an algorithm.
- represents the worst-case time complexity
- If a function $f(n)$ and $g(n)$ satisfy :
  There exists constants $c>0$ and $n_0$ such that for all $n \geq n_0$, $f(n) \leq c.g(n)$ then $f(n) \in O(g(n))$
- Example : let $f(n) = 3n + 2$ and $g(n) = n$, then $f(n) = O(n)$ since $f(n)$ grows linearly like $n$.



## 2. Omega Notation $\Omega(g(n))$

- describes the best-case performance.
- It gives a lower bound - the minimum time an algorithm will take.
- Let $f(n)$ and $g(n)$ be functions of $n$ where $n$ is steps required to execute programs $f(n) = \Omega g(n)$.
  There exist constant $c > 0$ and $n_0$ such that for all $n \geq n_0$. $f(n) \geq c.g(n)$

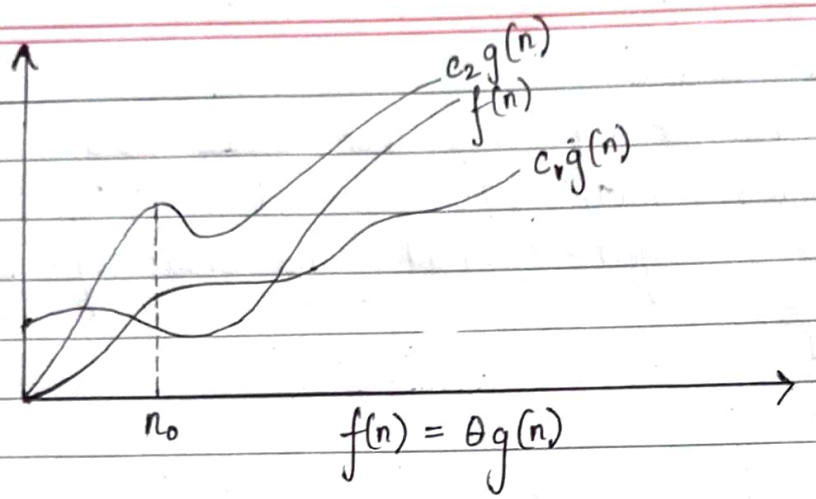
- Example: Let $f(n) = 3n+2$ and $3(n) = n$, then $f(n) = \Omega(n)$ because it grows at least like $n$




## 3. Theta Notation ($\theta$)

- describe the average-case or tight pound.
- It means the algorithm grows exactly at the same rate as $g(n)$ for large $n$
- If $f(n) = \theta(g(n))$, then there are constants $c_1$ and $c_2$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

- Example: If $f(n) = 3n+2$ and $g(n) = n$ then $f(n) = \theta(n)$




SYNTAX ERROR | Abhishek Rathor

These handwritten notes are exclusively created for educational purposes under the SYNTAX ERROR brand. Unathorized use, duplicated or redistribution in any form is strictly prohibited.

for collaboration or queries:

@ code.abhii07

# Basics of C++

## ⇒ C++ syntax

- C++ syntax refers to the set of rules that define how a C++ program is written and interpreted.
- C++ is case-sensitive, object-oriented, and statically-typed programming language.

Basic Structure:

```
# include < iostream >
using namespace std;
int main () {
    cout << " Hello, Everyone ! " << endl ;
    return 0 ;
}
```

- In C++, features like cout, cin, endl, string, etc., are part of the Standard Namespace (std).
- endl means end line. It moves the cursor to the next line (just like \n in C) and flushes the output buffer.
- Semicolon (;) – Every statement ends with a semicolon.

Comments:

```
// This is a single-line comment //
/* This is a multi-line comments */
```

## ⇒ C++ Data Types

C++ has two types of data types:

A. Primitive Data Types:

- byte (Not standard in C++, can use 'char' for 1 byte)
  Example: `char a = 100;`

- short - 2 bytes
  Example : `short b = 10000;`

- int - 4 bytes
  Example : `int c = 123456;`

- long - 4 or 8 bytes (depends on compiler)
  Example : `long d = 123456789L;`

- float - 4 bytes
  Example : `float e = 5.75f;`

- double - 8 bytes
  Example : `double price = 19.99;`

- char - 1 byte
  Example : `char g = 'A';`

- boolean - 1 byte (true or false)
  Example : `boolean isCodingFun = true;`

B. Non-Primitive Data Types :

- string - Represents a sequence of characters
  (e.g. `string name = "Coding";`)

- Array, Class, Struct, etc.

```
    getline (cin, name);
    cout << "Hello, " << name << "!" << endl;
    return 0;
}
```

Common Scanner Methods :

In C++, there's no scanner class. — instead, we use cin (for formatted input) and getline () (for full-line input)

| Method | Description |
|---|---|
| cin >> var | reads a single word or value (stops at space / newline.) |
| getline (cin, var) | reads an entire line including spaces. |
| cin >> intVar | reads an integer |
| cin >> doubleVar | reads a double value |
| cin >> boolVar | reads a boolean (0 for false, 1 for true) |

Note on Input Issues

In C++, after using cin >> for numbers, if you want to use getline () next, you must clear the newline character left in the buffer.

# C++ Loops (for, while, do-while)

## 1. Introduction to Loops

Loops in C++ are used to execute a block of code repeatedly until a specified condition is met.

## 2. for loop

Syntax:

```
for (initialization ; condition ; update) {
        // code to be executed
}
```

Example:

```
for (int i=1; i<=5 ; i++) {
    cout << " Count : " << i << endl;
}
```

Explanation:

starts with i=1
runs while i<=5.
prints value of i and increments i in each iteration

## 3. while loop

Syntax:

```
while (condition) {
        // code to be executed
}
```

Example:

```
int i = 1;
while (i <= 5) {
    cout << "Count : " << i << endl;
    i++;
}
```

Explanation:

initializes i = 1

prints value of i while i <= 5.

4. do-while loop

Syntax:

```
do {
    // code to be executed
} while (condition);
```

Example:

```
int i = 1;
do {
    cout << "Count : " << i << endl;
    i++;
} while (i <= 5);
```

Explanation:

executes the code at least once.

Then checks if i <= 5 and continues.

5. Special Case : do-while runs once

Example :

```
#include <iostream>
using namespace std;
int main () {
    int i = 10; // condition will be false initially
do {
    cout << "Runs once even if condition is false"
                                        << endl;
} while (i<5); // This is false, so loop
stops after first run

return 0;
}
```

Output : Runs once even if condition is false.

6. Summary Table

| | Loop type | Entry check | Runs at least once | Best when |
|---|---|---|---|---|
| a) | for | Yes | No | Iterations known |
| b) | while | Yes | No | Conditions-based loop |
| c) | do-while | No | Yes | run once before checking condition |

# C++ Conditional Statements (if-else, switch)

## 1. Introduction to Conditional Statements

Conditional statements in C++ allow the program to take decisions based on conditions.

There are two primary types:

- if-else
- switch

## 2. if-else Statement

Syntax:

```
if (condition) {
        // code if condition is true
} else {
        // code if condition is false
}
```

Using else-if for multiple conditions:

```
if (condition1) {
        // code block
} else if (condition2) {
        // code block
} else {
        // default code
}
```

Example:

```
#include <iostream>
using namespace std;
int main() {
```

```
int number = 10;
if (number > 0) {
    cout << "Positive number" << endl;
} else {
    cout << "Non-positive number" << endl;
}
return 0;
}
```

Explanation:
Checks if number is greater than 0 and prints the appropriate message based on the condition.

3. switch Statement
used when you have multiple conditions based on the same variable.

Syntax:

```
switch (expression) {
    case value 1:
        // code
        break;
    case value 2:
        // code
        break;
    default:
        // default code
}
```

Example:

```
#include <iostream>
using namespace std;
int main() {
    int day = 2;
    switch (day) {
        case 1:
            cout << "Monday" << endl;
            break;
        case 2:
            cout << "Tuesday" << endl;
            break;
        default:
            cout << "Invalid day" << endl;
    }
    return 0;
}
```

Explanation:

- Check the value of 'day'
- executes the matching case and exists with break
- If no case matches, runs the default case.

4. Key Differences : if-else v/s Switch

- If-else works with boolean expressions, while switch compares a single variable to multiple values.
- Switch is cleaner for multiple fixed values.
- If-else is more flexible and supports ranges and complex conditions.

# Functions & Recursion in C++

A function (method) is a block of code that performs a specific task. It increases code reusability and improves readability.

Why we use Functions ?

- Avoid code repetition
- Enhance modularity
- Improve debugging and testing
- Easy collaboration in large codebases

Syntax :

```
<return_type> functionName (<parameters>) {
// code block
  return value;
}
```

Example :

```
#include <iostream>
using namespace std;
int add (int a, int b) {
    return a+b;
}
int main () {
    cout << "Sum :" << add (5,3) << endl;
    return 0;
}
```

Types of Functions

1. Predefined (library) Functions

   Examples :
   - max (a,b) → from <algorithm>
   - sqrt (x) → from <cmath>
   - cout << → from <iostream>

## 2. User-defined Functions
custom functions created by the programmer.

Function Example

```
#include <iostream>
using namespace std;

// Function definition
int multiply (int a, int b) {
        return a * b;
}
int main () {
    int result = multiply (5,6);
    cout << "Multiplication is : " << result << endl;
    return 0;
}
```

- Time Complexity : O(1)
- Space Complexity : O(1)

## Recursion :

Definition — Recursion is a process where a function calls itself to solve smaller sub-problems of a main problem.

Syntax :

```
return_type functionName (parameters) {
    if (base_condition) {
        return result;
    }
    return functionName (modified_parameters);
}
```

## Example 1 : Factorial

```
# include < iostream >
using namespace std ;
int factorial (int n) {
    if (n == 0 || n == 1)
        return 1;
    return n * factorial (n-1) ;
}
int main () {
    int num = 5;
    cout << "Factorial of " << num << " is " << factorial (num)
                                                << endl;
    return 0;
}
```

Dry Run : Factorial (3)

factorial (3) = 3 * factorial (2)
= 3 * 2 * factorial (1)
= 3 * 2 * 1.
= 6

- Time Complexity : O(n)
- Space Complexity : O(n)

## Example 2 : Fibonacci

```
# include < iostream >
using namespace std.
int fibonacci (int n) {
    if (n == 0) return 0 ;
    if (n == 1) return 1;
    return fibonacci (n-1) + fibonacci (n-2) ;
}
```

```
int main () {
    int num = 6.
    cout << " Fibonacci of " << num << " is " << fibonacci (num)
                                                << endl;
    return 0;
}
```

- Time Complexity : $O(2^n)$
- Space Complexity : $O(n)$

# Recursion Vs Iteration

Recursion
- use function calls
- High memory (stack)
- slower due to overhead.

Iteration
- use loops
- low memory
- faster execution

→ When we should use Recursion?

- Divide - and - conquer algorithms (e.g. Merge Sort)
- Tree traversal (DFS)
- Backtracking (e.g. N-Queens)
- Dynamic Programming (use memorization)

→ Common Recursion Mistakes

- Missing base case
- Incorrect input modification
- Incorrect return statements.

## Imp Interview Questions

Factorial
Fibonacci
Reverse a string
Palindrome check
Sum of digits
Tower of Hanoi
Subsets generation
N-Queens problems
Tree traversal

# Arrays & Strings in C++

⇒ **Array in C++**

Definition: An array is a data structure that stores a fixed-size sequential collection of elements of the same type.

→ Why use Array?
- Efficient memory usage
- Easy element access
- suitable for static data storage

→ Syntax:

```
<datatype> arrayName[size];
```

→ Example:

```
int arr[5];      // declare an array of size 5.
arr[0] = 10;     // assign value
```

→ Initialization

```
int arr [] = { 1, 2, 3, 4, 5 }
```

→ Traversal Example

```
int main () {
    int arr [] = {1,2,3,4,5};
    int size = sizeof (arr) / sizeof (arr [0]);
    for (int i=0 ; i< size ; i++) {
        cout << arr [i] << endl;
    }
    return 0;
}
```

→ Time Complexity

- Access : O(1)
- Search : O(n)
- Insertion / Deletion : O(n)

→ Space Complexity : O(n)

⇒ Strings in C++

Definition : A string is an object that represents a sequence of characters.

→ Why Strings are special ?

- Strings in C++ are mutable (can be changed)
- stored dynamically (unlike fixed-size character arrays)
- Provide many built-in functions for manipulation.

→ Declaration & Initialization

```
string str1 = "Hello";
string str2("World");
```

→ Concatenation

```
string result = str1 + " " + str2;
```

→ Common String Methods:

| | Description |
|---|---|
| str.length() | → returns string length |
| str[i] | → access character at index |
| str.substr(start, length) | → returns substring |
| str.find("abc") != string::npos | → checks if substring exists |
| str == "abc" | → string comparison |
| str.c_str() | → converts to C-style char array |

→ Example Code:

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s = "Syntax Error";
    cout << "Length: " << s.length() << endl;
    cout << "Character at index 2: " << s[2] << endl;
    cout << "Substring (0,6): " << s.substr(0,6) << endl;
    if (s.find("Error") != string::npos)
        cout << "Contains error" << endl;
    if (s == "Syntax Error")
        cout << "String equals 'Syntax Error'"
             << endl;
    return 0;
}
```

→ Time Complexity
- Access : O(1)
- Concatenation (naive) : O(n)
- Searching substring : O(n * m) where n = text length, m = pattern length

→ Space Complexity : O(n)

## Common Interview Questions

Array —
- Find max/min in array
- Reverse an array
- Find duplicate elements
- Kadane's Algorithm (Maximum subarray sum)

String —
- Reverse a string
- Palindrome check
- Anagram check
- Longest substring without repeating characters.

Example : Reverse a String

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s = "Hello";
    string reversed = "";
    for (int i = s.length() - 1; i >= 0; i--) {
        reversed += s[i];
}
```

```
    cout << "Reversed string:" << reversed << endl;
    return 0;
}
```

→ Time Complexity
- Naive string ..... concatenation : $O(n^2)$
- Optimized (using "ostringstream" or "string builder" style) : $O(n)$

Example : Reverse an Array

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {1,2,3,4,5};
    int n = sizeof(arr) / sizeof(arr[0]);
    int start = 0, end = n-1;
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
    cout << "Reversed array :";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```

→ Time complexity : $O(n)$
→ Space Complexity : $O(1)$

# Arrays (1D and 2D), Operations

## ⇒ 1D Arrays

Definition → A 1D array is a linear structure that holds a fixed number of values of a single data type.

Declaration in C++ →

```
// Declaration + Memory Allocation
int arr[5];
// Direct initialization
int arr[] = {1,2,3,4,5};
```

Real-life Example → Storing marks of 5 students :

```
int marks[5] = { 90, 85, 75, 92, 80};
```

## ⇒ 2D Arrays

Definition → A 2D Array is an array of arrays. (like a matrix)

Declaration in C++ →

```
// Declaration + Memory allocation (3x3 matrix)
int matrix[3][3];

// Direct initialisation
int matrix[3][3] = {
        {1,2,3}, {3,4,5}, {5,6,1} };
```

Example Use → used to represent tables, matrices, or grids in games (like a chessboard, tic-tac-toe).

## ⇒ Array Operations

1. Insert :

Insert at index (manually shift elements)

```
// shifting elements to right
for (int i=n; i > pos; i--) {
        arr[i] = arr[i-1];
}
arr[pos] = value;
n++;
// Print array
for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
}
```

2. Delete :

Delete (element at index) at index

```
// shift left
for (int i = pos; i<n-1; i++) {
        arr[i] = arr[i+1];
}
n--;
// Print array
for (int i=0; i<n; i++)
        cout << arr[i] << " ";
}
```

3. Update :

```
arr [index] = 99;  // update value at index
for (int i=0; i<5; i++)
    cout << arr [i] << " ";
}
```

4. Prefix Sum

⇒ Definition : Prefix sum is a technique to preprocess an array so that range sum queries can be answered in O(1).

⇒ Example :

Original : [2, 3, 5, 11]

Prefix : [2, 5, 10, 11]

⇒ Code :

```
prefix [0] = arr [0];
for (int i = 1; i< n ; i++) {
    prefix [i] = prefix [i-1] + arr [i];
}
```

5. Sliding Window

⇒ Definition : A technique to work on contiguous subarrays of a fixed or variable size — optimizing brute force solutions.

⇒ Use Case : Max sum of subarray of size k.

⇒ Code :

```
int maxSum (vector <int> & arr , int k) {
    int n = arr. size ();
```

```
    if (n < k) return -1;  // not possible
int windowSum = 0;
for (int i=0; i<k; i++)
    windowSum += arr[i];
int maxSum = windowSum;
for (int i=k; i<n; i++) {
    windowSum += arr[i] - arr[i-k];  //slide window
    maxSum = max(maxSum, windowSum);
}
return maxSum;
}
```

6. Kadane's Algorithm

⇒ Definition → used to find the maximum subarray sum in O(n) time.

⇒ Key Idea → Track 'currentSum', if it goes below 0 → reset it.

⇒ Code →

```
int maxSubArray(vector<int>& arr) {
    int n = arr.size();
    int maxSum = arr[0];
    int currSum = arr[0];
    for (int i=1; i<n; i++) {
        currSum = max(arr[i], currSum + arr[i]);
        maxSum = max(maxSum, currSum);
    }
    return maxSum;
}
```

# Sorting Algorithms (C++ code + Theory)

## 1. Bubble Sort

Theory : repeatedly swap adjacent elements if they are in wrong order. Ineffecient but easy.

- Time : $O(n^2)$
- Best : $O(n)$ (Already sorted)

## 2. Selection Sort

Theory : Find the minimum element and place it at the beginning. Repeat for the rest.

- Time : $O(n^2)$
- Space : $O(1)$

## 3. Insertion Sort

Theory : Take an element and insert it into the correct position in the sorted left part.

- Time : $O(n^2)$
- Best : $O(n)$
- Use case : Small datasets

## 4. Merge Sort

Theory : Divide array into halves, sort each half, then merge. It's a divide and conquer algorithm.

- Time : $O(n \log n)$
- Space : $O(n)$

@ SYNTAX ERROR
— Abhishek Rathore

## 5. Quick Sort

Theory: Pick a pivot and partition the array such that left < pivot < right. Then sort both halves recursively.

- Time: $O(n \log n)$ (average), $O(n^2)$ (worst)
- Space: $O(\log n)$

## 6. Counting Sort

Theory: count frequency of elements and reconstruct the sorted array.
works only for non-negative integers.

- Time: $O(n+k)$ where $k$ = max element
- Space: $O(k)$

## #1 Summary Table

| Algorithm | Time Complexity | Space | Stable | Use Case |
|---|---|---|---|---|
| Bubble Sort | $O(n^2)$ | $O(1)$ | Yes | Teaching Basics |
| Selection Sort | $O(n^2)$ | $O(1)$ | No | Memory-limited |
| Insertion Sort | $O(n^2)$, Best $O(n)$ | $O(1)$ | Yes | Nearly sorted arrays |
| Merge Sort | $O(n \log n)$ | $O(n)$ | Yes | Large datasets |
| Quick Sort | $O(n \log n)$, Worst $O(n^2)$ | $O(\log n)$ | No | Performance critical apps |
| Counting Sort | $O(n+k)$ | $O(k)$ | Yes | Bounded range & integers |

# Strings

⇒ Definition : A string is a sequence of characters. In C++, strings are mutable (can be changed).

⇒ Common Operations :

```
string rev = s; reverse (rev.begin (), rev.end ()); // reverse
    string s = "Hello";
int len = s.length ();          // length of string
 char ch = s.at (1);            // access character at index 1
string sub = s.substr (1,3);    // from index 1, length 3
string upper = s;               // convert to uppercase
for (char &c : upper) c = toupper (c);
 string s2 = s + " World";      // concatenation
  s.append ("world");           // append.
```

## Palindrome Check

⇒ Definition : A string is a palindrome if it reads the same forward and backward.

⇒ Example : "madam", "racecar".

⇒ Code :

```
bool isPalindrome (string s) {
    int i = 0, j = s.length () - 1;
    while (i < j) {
        if (s[i] != s[j]) return false;
        i++;
        j--;
    }
    return true;
}
```

# Anagram Check

⇒ Definition: Two strings are anagrams if they contain the same characters in any order.

⇒ Example: "listen" and "silent"

⇒ Code:

```
bool isAnagram (string s1, string s2) {
    if (s1.length() != s2.length()) return false;

    // Sort both strings
    sort(s1.begin(), s1.end());
    sort(s2.begin(), s2.end());
    return s1 == s2;
}
```

# Pattern Matching Algorithms

1. KMP (Knuth - Morris - Pratt)
   - Idea: Preprocesses pattern to avoid redundant comparisons
   - Use case: Effecient pattern searching in large texts
   - Time Complexity: O(n+m)
     (n = length of text, m = length of pattern)

2. Rabin - Karp
   - Idea: use rolling hash function to compare substrings.
   - Use Case: Multiple pattern matching, plagiarism detection.
   - Time Complexity:
     - Average case: O(n+m)
     - Worst case: O(nm) (when many hash collisions occur)

# Substrings

⇒ Definition : A substring is a contiguous sequence of characters of characters within a string.

⇒ Count of all substrings : For string of length n, total substrings = $n(n+1)/2$

• Functions to generate all substrings, count unique substrings, and find longest common substring.

⇒ Code to Print All Substrings :

```
for (int i=0; i< str. length (); i++) {
    for (int j=1; j < str. length (); j++) {
        cout << str. substr (i, j-i+1) << endl;
    }
}
```

# Longest Common Prefix

⇒ Definition : The longest starting portion of all strings that is common.

⇒ Use Case : Auto-suggestions, command-line tools, dictionary search

⇒ Code :

```
string longestCommonPrefix (vector < string > & strs) {
    if (strs. empty ()) return "";
    string prefix = strs [0];
    for (int i = 1; i< strs. size (); i++) {
        while (strs [i]. find (prefix) != 0) {
            prefix = prefix. substr (0, prefix. size () - 1);
```

```
        if (prefix.empty()) return "";
    }
}
return prefix;
}
```

# Summary Table

| Topic | Use Case | Time Complexity |
|---|---|---|
| Palindrome Check | validation, puzzles | O(n) |
| Anagram Chack | word comparison | O(n) |
| KMP Algorithm | pattern searching | O(n+m) |
| Rabin-Karp Algorithm | plagiarism check, combinational prblms | avg: O(n+m)<br>O(n²) |
| Substrings | auto-complete, search suggestions | O(nm). |
| Longest Common Prefix | | |

# Searching & Binary Search

## # Linear Search

Def: Checks each element one by one to find the target.

Use Case: When the array is unsorted or small.

Code:
```
int linearSearch(int arr[], int n, int target) {
    for (int i=0; i<n; i++) {
        if (arr[i] == target)
            return i;    // element found
    }
    return -1;    // element not found.
}
```

Important Table

| Case | Time Complexity | Space Complexity | Explanation |
|---|---|---|---|
| Best case | O(1) | O(1) | Element found at the first index |
| Average case | O(n) | O(1) | Element found some-where in the middle |
| Worst case | O(n) | O(1) | Element not present or found at last index |

Notes:

- n = number of elements in the array.
- Space Complexity is O(1) because Linear Search uses no extra space, only a few variables for indexing/comparisons.

# Binary Search

Def : Divide and conquer : check the mid element and search in the appropriate half.

Use Case : Effecient search in sorted arrays.

Code :
```
int binarySearch (int arr [], int n, int target) {
    int low = 0, high = n-1;
    while (low <= high) {
        int mid = low + (high-low)/2;
        if (arr [mid] == target)
            return mid; // element found
        else if (arr [mid] < target)
            low = mid + 1; // search right half
        else
            high = mid - 1; // search left half
    }
    return -1; // element not found
}
```

## Important Table

| Case | Time Complexity | Space Complexity (Recursive) | Space Complexity (Iterative) |
|---|---|---|---|
| Best case | O(1) | O(1) | O(1) |
| Average case | O(log n) | O(log n) | O(1) |
| worst case | O(log n) | O(log n) | O(1) |

Notes :

- n = number of elements in the sorted array
- Iterative version uses O(1) space because it only uses loop variables.
- Recursive version uses O(log n) space due to the recursive call stack.

# Lower Bound

Def → The first index where the element is greater than or equal to the target.

Code →

```
int lowerBound (int arr[], int n, int target) {
    int low = 0, high = n;
    while (low < high) {
        int mid = (low+high) /2;
        if (arr[mid] < target)
            low = mid+1;
        else
            high = mid;
    }
    return low;
}
```

UseCase → frequency and range queries.

Time Complexity → O(log n)

Array Type → Sorted

# Upper Bound

Def → The first index where the element is strictly greater than the target.

Code →
```
int upperBound(int arr[], int n, int target) {
    int low = 0, high = n;
    while (low < high) {
        int mid = (low + high)/2;
        if (arr[mid] <= target)
            low = mid + 1;
        else
            high = mid;
    }
    return low;
}
```

Use case → frequency and range queries

Time Complexity → $O(\log n)$

Array Type → Sorted

# Search in Rotated Sorted Array

Definition → Array is sorted but rotated. Find the index of the target efficiently.

Idea → Find which half is sorted and decide which part to discard.

Code →

```
int searchRotated(int arr[], int n, int target) {
    int low = 0, high = n-1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target)
            return mid;
        // Left half is sorted
        if (arr[low] <= arr[mid]) {
            if (arr[low] <= target && target
                < arr[mid])
                high = mid - 1;
            else
                low = mid + 1;
        }
        // Right half is sorted
        else {
            if (arr[mid] < target && target
                <= arr[high])
                low = mid + 1;
            else
                high = mid - 1;
        }
    }
    return -1; // not found
}
```

Time Complexity : $O(\log n)$

# Binary Search on Answer Problems

Definition → Apply binary search on a range of answers instead of the array itself.

Use Case → When you're optimizing or finding min/max values like:

- Minimum days to complete tasks
- Maximum pages read per day
- Aggressive cows, Painters partition, etc.

Structure Template →

```
int binarySearchAnswer(int arr[], int n) {
    int low = min_possible, high = max_possible;
    int ans = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (isValid(mid)) {
            ans = mid;
            high = mid - 1;  // or low = mid + 1 (based on prblm)
        }
        else {
            low = mid + 1;  // or high = mid - 1 (based on prblm)
        }
    }
    return ans;
}
```

Important → Define the isValid() function based on the problem.

# Recursion & Backtracking

⇒ What is Recursion ?

When a function calls itself to solve smaller sub-problems.

Structure :

```
void function () {
    // base case
    // recursive call (s)
}
```

Key Concepts :
- Base case : stops the recursion
- recursive case : breaks the problem into smaller pieces.

Factorial using Recursion :

```
int factorial (int n) {
    if (n==0 || n==1) return 1;   // base case
    return n * factorial (n-1);   // recursive call
}
```

Time Complexity : $O(n)$

Fibonacci using Recursion :

```
int fibonacci (int n) {
    if (n <=1) return n;
    return fibonacci (n-1) + fibonacci (n-2);
}
```

Time Complexity : $O(2^n)$ → use memoization for optimi-zation.

Note : not effecient for large n.

# Subsets (Power Set)

Definition : generate all possible subsets of a given array/ string.

Code :
```
void generateSubsets (vector<int> & arr, int index,
vector <int> & current, vector <. vector <int>> &
    result) {
  if (index == arr.size ()) {
        result. push_back (current);
        return;
  }
  // Include
  current. push_back (arr [index]);
  generateSubsets (arr, index+1, current , result);

  // Exclude
  current. pop_black ();
  generate Siebsets (arr, index+1, current , result);
}
```

⇒ What is Backtracking ?

Definition → Try all possible options and backtrack (undo the choice) if a solution is not valid.

Use case → Solving constraint-based problems like puzzles, paths, arrangements.

@ SYNTAX ERROR
— Abhishek Rathor

→ Steps in Backtracking —
1) Choice
2) Explore
3) Unchoose (Backtrack)

# N-Queens Problem

Definition → Place N queens on an N×N chessboard such that no two queens attack each other.

Uses backtracking :
1. Choice → Place a queen in a column
2. Explore → Move to the next row.
3. Unchoose (Backtrack) → Remove queen if it leads to no solution.

Code Snippet :

```
bool isSafe (vector<string> &board, int row, int col, int n) {
    for (int i=0; i<row; i++) {
        if (board[i][col] == 'Q')
            return false;
    }
    for (int i = row-1, j = col-1; i>=0 && j>=0; i--, j--) {
        if (board[i][j] == 'Q')
            return false;
    } ....
void solve (vector<string> &board, int row, int n, vector<vector<string>>
                                                    &result)
    if (row == n) {
        result.push_back (board);
        return;
    }
```

```
for (int col = 0; col < n; col++) {
        if (isSafe(board, row, col, n)) {
                board[row][col] = 'Q'; // Place queen
                solve(board, row + 1, n, result);
                board[row][col] = '.'; // Backtrack
        }
    }
}
vector<vector<string>> solveNQueens(int n) {
        vector<vector<string>> result;
        vector<string> board(n, string(n, '.'));
        solve(board, 0, n, result);
        return result;
}
```

⇒ Time Complexity : $O(N!)$

# Rat in Maze

Definition → From top-left to bottom right, move in all directions to find all paths.

Allowed Moves : Down, Right, Up, Left
We need to print all possible paths.

Code Snippet :

```
// Backtracking function
void solveMaze(vector<vector<int>>& maze, int x, int y,
        vector<vector<int>>& visited, string path, vector<string>
        & result, int n) {
// If destination is reached
    if (x == n-1 && y == n-1) {
            result.push_back(path);
            return;
    }
```

```
// Move down
if (isValid(x+1, y, maze, visited, n)) {
    solveMaze(maze, x+1, y, visited, path + 'D', result, n);
}
// Same for Right, Up, Left ....
}
```

→ Time Complexity : $O(4^{n*n})$ worst-case

# Sudoku Solver

Definition → Fill a 9x9 board so that each row, column, and 3x3 grid has digits 1-9.

Code Snippet :

```
bool isValid(vector<vector<char>>& board, int row, int col,
             char num) {
    // check row, col, and 3x3 grid
}
bool solveSudoku(vector<vector<char>>& board) {
    for (int row = 0; row < N; row++) {
        for (int col=0; col < N; col++) {
            if (board[row][col] == '.') { // empty cell
                for (char num = '1'; num <= '9'; num++) {
                    if (isValid(board, row, col, num)) {
                        board[row][col] = num; // place number
                        if (solveSudoku(board)) return true;
                        board[row][col] = '.'; // backtrack
                    }
                }
                return false; // no valid number found
            }
        }
    }
    return true; // solved
}
```

→ Time Complexity : $O(9^{81})$ worst case

## Summary Table

| Problems | Type | Technique | Time Complexity |
|---|---|---|---|
| Factorial | Recursion | Base case + call | $O(n)$ |
| Fibonacci | Recursion | Tree Recursion | $O(2^n)$ |
| Subsets | Recursion | Include / Exclude | $O(2^n)$ |
| N-Queens | Backtracking | Safe Placing | $O(N!)$ |
| Rat in Maze | Backtracking | Move in 4 dirs | $O(4^{n*n})$ |
| Sudoku Solver | Backtracking | Fill + Undo | $O(9^{81})$ |

SYNTAX ERROR | Abhishek Eswar

These handwritten notes are exclusively written for education purposes under the SYNTAX ERROR brand.
Unauthorized use, duplicated or redistribution in any form is strictly prohibited.
for collaboration or queries : @ code . abhii 07

# Hashing

Definition → Hashing maps data (keys) to a fixed-size value (index) using a hash function.

- unordered_map → stores key-value pairs.
- unordered_set → stores only unique values.

Average time complexity for insertion, deletion, and lookup : O(1)

## 1. Unordered_map

Syntax :

```
int main () {
    unordered_map <string, int> map; // declare an unordered_map
    map ['apple'] = 3;
    map ['banana'] = 5;
    cout << 'apple count :' << map ['apple'] << endl;
    // check if key exists
    if (map.find ('apple') != map.end ())
        cout << 'apple exists' << endl;
    map.erase ('apple');
    return 0;
}
```

## 2. Unordered_set

Syntax :

```
int main () {
    unordered_set <int> set; // declare unordered set
    set.insert (5);
    set.insert (10);
    if (set.find (5) != set.end ())
        cout << '5 exists' << endl;
    set.erase (5);
    return 0;
}
```

## # Frequency Counting using unordered_map

- Problem: count how many times each element appears
- Use case: count occurence.
- Code:

```
int main () {
    vector <int> arr = {1, 2, 3, 2, 1, 4, 2}
    unordered_map < int, int > freq;
    // count occurence
    for (int num : arr) {
        freq[num]++;
    }
    // Print frequencies
    for (auto &p : freq) {
        cout << p.first << '→' << p.second << endl;
    }
    return 0;
}
```

Time Complexity : O(n)

## # Two Sum Problem

- Problem: Find indices of two numbers in an array that add up to a target.
- Input: arr = {2, 7, 11, 15}, target = 9
- Output: (0, 1) because 2+7 = 9.

- Code:

```
vector <int> twoSum (vector<int>& nums, int target) {
    unordered_map < int, int > map;
    for (int i=0; i< nums.size (); i++) {
        int diff = target - nums [i];
        // check if complement exists
```

```
        if (map.find(diff) != map.end()) {
            return {map[diff], i}; // return indices
        }
        // Store current number with index
        map[nums[i]] = i;
    }
    return {-1, -1}; // if no solution.
}
```

Time Complexity : O(n)
Use case : pair sum equals target

# Subarrays with Given Sum (Positive Integers Only)

- Problem : Count the number of subarrays whose sum equals a given target k.
- Prefix sum + unordered_map : Technique
- Code :

```
int subarraySum(vector<int>&nums, int k) {
    unordered_map<int, int> map; // prefixsum → frequency
    map[0] = 1; // base case (empty prefix)
    int sum = 0, count = 0;
    for (int num : nums) {
        sum += num;
        // If (sum-k) exists, it means we found a subarray
        if (map.find(sum-k) != map.end()) {
            count += map[sum-k];
        }
        // Store / update current prefix sum
        map[sum]++;
    }
    return count;
}
```

- Time Complexity : O(n)

# Interview - Level Unordered Problems

## # Longest Consecutive Sequence

- Problem : Find the length of the longest consecutive elements sequence.
- Code :

```
int longestConsecutive (vector <int> & nums) {
    unordered_set <int> set;
    for (int num : nums) {
        set. insert (num);
    }
    int maxlen = 0;
    for (int num : set) {
        if (set. find (num-1) == set. end()) {
            int curr = num;
            int len = 1;
            while (set. find (curr + 1) != set. end()) {
                curr++;
                len++;
            }
            maxlen = max (maxlen, len);
        }
    }
    return maxlen;
}
```

- Time Complexity : O(n)
- Use Case : continuous elements

# Group Anagrams

- Problem : Group strings that are anagrams.
- Code :

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> map;
    for (string s: strs) {
        string key = s;
        sort(key.begin(), key.end()); // sort chars to form key
        map[key].push_back(s);
    }
    vector<vector<string>> result;
    for (auto& entry : map) {
        result.push_back(entry.second);
    }
    return result;
}
```

- Time Complexity : $O(n * k \log k)$
  ($n$ = no. of words, $k$ = avg length of word)
- Use Case : Anagram grouping.

# Stack & Queue

## Stack :

Definition → A stack is a linear data structure that follows that LIFO (Last In First Out) principle. Stack has one end for insertion & deletion (called the top of the stack).

A stack is a container in which insertion & deletion can be done from the end known as the top of the stack.

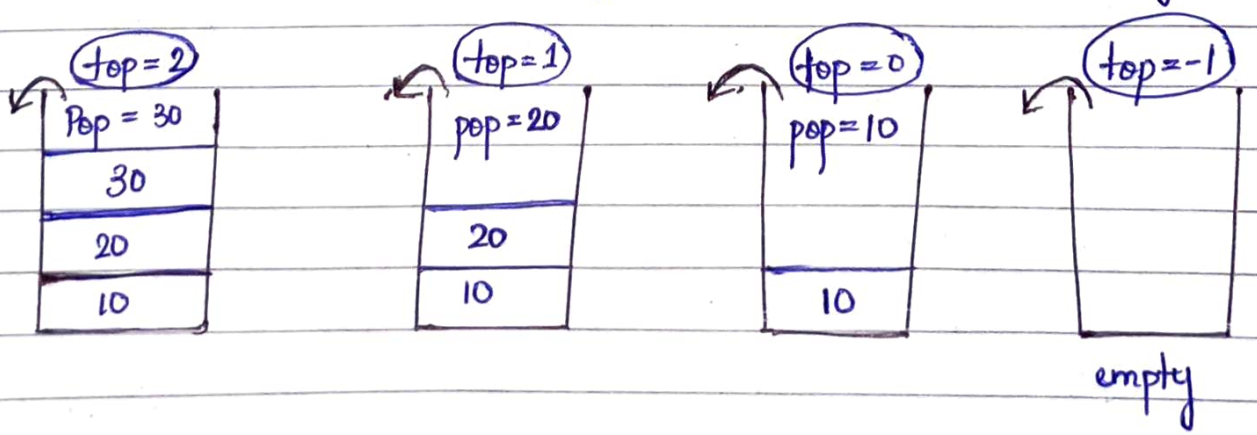
A stack is an abstract Data Type with a pre-defined capacity, which means it can store elements of limited size.

## Operations on the Stack –

1) Push() : When we insert an element in a stack then the operation is known as push. If stack is full, overflow condition occurs.

2) Pop() : When we delete an element from stack, the operation is called as pop(). If stack is empty means no element exists in the stack, this state is known as underflow state.

3) Peek() : It returns the top element without removing it.



# Implementation of Stack using Array

```
class MyStack {
        int * stack;
        int top;
        int size;
public:
    MyStack (int size) {
        this -> size = size;
        stack = new int [size];
        top = -1;       // initially empty
    }
    void push (int val) {
        if (top == size-1) {
            cout << "Overflow" << endl;
            return;
        }
        stack [++top] = val;
        cout << val << "pushed into stack" << endl;
    }
    int pop() {
        if (top == -1) {
            cout << "Underflow" << endl;
            return -1;
        }
        return stack [top--];
    }
    int peek() {
        if (top == -1) {
            cout << "Stack is empty" << endl;
            return -1;
        }
```

```
        return stack[top];
    }
    MyStack() {
        delete[] stack;
    }
};
```

## Implementation of Stack using Linked list

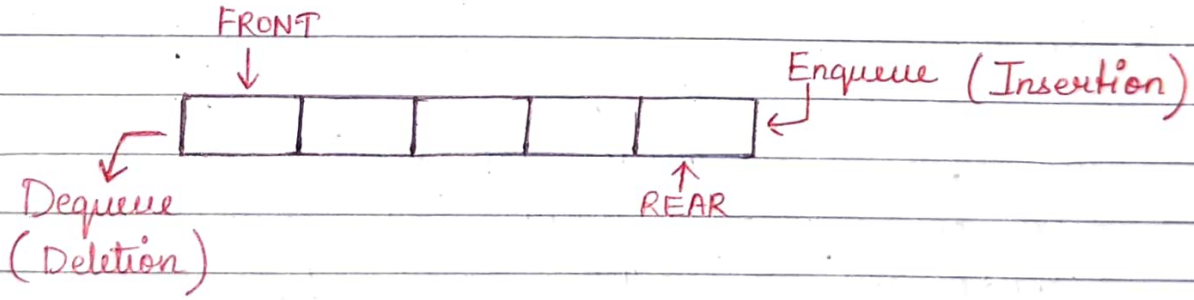
```
class Node {
public:
    int data;
    Node * next;
    Node(int data) {        // constructor
        this -> data = data;
        this -> next = nullptr;
    }
};
// stack class using Linked list
class StackLL {
    Node* top;              // pointer to top of stack
public:
    // constructor
    StackLL() {
        top = nullptr;
    }
int pop() {                 // Pop operation
    if (top == nullptr) {
        cout << "Stack Underflow!." << endl;
        return -1;
    }
```

```
// Peek operation
int peek () {
    if (top == nullptr) {
        cout << "Stack is empty." << endl;
        return -1;
    }
```

# Queue

A queue can be defined as ordered list which enables insert operations to be performed at one end called REAR and delete operations to be performed at another end called FRONT.

→ Queue can be referred as to be first In First out list (FIFO).




## Complexity of Queue —

| | | |
|---|---|---|
| Queues | Average<br>Access, Search, Deletion, Insertion<br>O(n) O(n) O(1) O(1) | Space Complexity - average<br>O(n) |
| Queue | Worst<br>Access, Search, Insertion, Deletion<br>O(n) O(n) O(1) O(1) | Space Complexity worst<br>O(n) |

## Operation on Queue –

1) Enqueue – Enqueue is used to insert element at rear end of the queue. It return void.

2) Dequeue – Dequeue operations performs the deletion from front end of queue. The deque operation can also be designed to void.

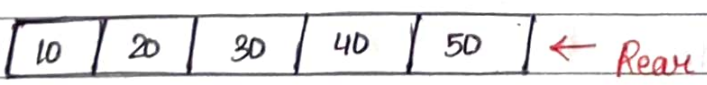
3) Peek – This returns element which is pointed by front pointer in the queue but does not delete.

4) Queue overflow – (is Full) – when queue is completely full, then it shows overflow condition.

5) Queue underflow (is Empty) – when there is no element in the queue then it throws underflow condition.

## Types of Queue –

(1) Linear queue – In linear queue, an insertion takes place from one end while deletion takes place from another end. It strictly follows FIFO Rule. The linear queue can be represented as shown –




- Elements are added at the rear end.
- Elements are removed from the front end.
- When we keep adding elements, the rear moves forward.

## Drawbacks of linear Queue

- Insertion can only happen at the rear.
- Once the queue is full (rear reaches the last index), new elements cannot be added, even if there are empty spots at the front due to deletion.
- This leads to a waste of space and is called the overflow condition, even though the queue might not be truly full.

② Circular Queue – In a circular queue, the last position is connected back to the first position, forming a circle.

- It works like a linear queue but makes better use of space.
- When the rear reaches the end, it goes back to the beginning if there's space.
- This helps to reuse the freed space at the front after deletions

Drawbacks of linear queue is overcomed in circular queue. If empty space is available, new element can be added to empty space by simply incrementing value of rear.

③ Priority queue – The queue in which element has some priority associated with it based on priority of the elements, elements are arranged in a priority queue. If elements occurs with same priority, then they are served according to FIFO principle.

# Queue Implementation using Array

```
#include <iostream>
using namespace std;

class MyQueue {
        int * queue;
        int front, rear, size;
public:
    MyQueue (int s) {
            size = s;
            queue = new int [size];
            front = 0;
            rear = 0;
        }
    void enqueue (int val) {
            if (rear == size) {
                cout << "Queue Overflow \n";
                return;
            }
            queue [rear++] = val;
        }
    int dequeue () {
        if (front == rear) {
            cout << "Queue Underflow \n";
            return -1;
        }
        return queue [front++];
    }
    int peek () {
        if (front == rear) {
            cout << "Queue is Empty \n";
            return -1;
        }
        return queue [front];
} }
```

@ SYNTAX ERROR
— Abhishek Error

## Queue Implementation using Linked list

```
class Node {
public :
    int data;
    Node * next;
    Node (int val) {
        data = val;
        next = NULL;
    }
};
class QueueLL {
        Node * front;
        Node * rear;
public :
    QueueLL () {
        front = NULL;
        rear = NULL;
    }
    void enqueue (int val) {
        Node * newNode = new Node (val);
        if (rear == NULL) {
            front = rear = newNode;
        } else {
            rear -> next = new Node;
            rear = newNode;
        }
    }
    int dequeue () {
        if (front == NULL) {
            cout << " Queue Underflow \n";
            return -1;
        }
```

```
        int val = front -> data;
        Node* temp = front;
        front = front -> next;
        if (front == NULL) rear = NULL;
        delete temp;
        return val;
    }
};
```

## Inbuilt Stack & Queue in C++ (STL)

```
int main() {
    queue <int> q;
    q.push(5);
    q.push(10);          // enqueue
    cout << q.front() << endl;  // peek -> 5.
    q.pop();             // dequeue (removes 5)
    cout << q.front() << endl;  // now front -> 10.
    return 0;
}
```

## Prefix, Infix & Postfix Expressions

- Infix : A + B → Human-readable
- Postfix : AB + → Evaluated using stack
- Prefix : + AB → Also evaluated using stack.

## Evaluate Postfix Expression

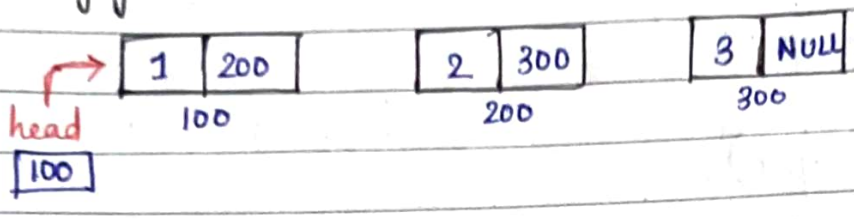
```
int evaluatePostfix (string expr) {
    stack <int> st;
    stringstream ss (expr);
    string token;

    while (ss >> token) {
        if (token == '+' || token == '-' || token == '*' ||
            token == '/') {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (token == '+') st.push (a+b);
            else if (token == '-') st.push (a-b);
            else if (token == '*') st.push (a*b);
            else if (token == '/') st.push (a/b);
        }
        else {
            st.push (stoi (token)); // convert string → int
        }
    }

    return st.top();
}
```

## Do it Yourself

Monotonic Stack / Queue Famous Problems.

- Next Greater Element
- Next Smaller Element
- Sliding window
- Histogram problems

# 1. Singly Linked List




Example –

```
class Node {
    public:
        int data;
        Node* next;
        Node(int data) {
            this -> data = data;
            this -> next = NULL;
        }
};
```

Explanation –

```
Node* head = new Node(10);
head -> next = new Node(20);
head -> next -> next = new Node(30);
```

→ Each node stores the value and a reference (pointer) to the next node.
→ Traversing from head goes through all elements.

# 2. Doubly Linked List

- Each node has three parts :

a. prev → link to the previous node
b. data → value
c. next → link to the next node

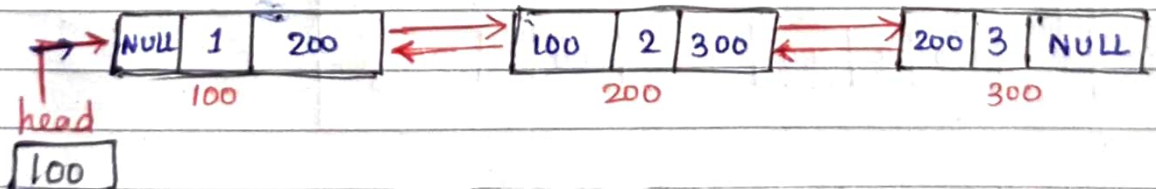
- we can move both forward and backward.

Example –

```
class Node {
    public :
        int data;
        Node * prev;
        Node * next;
        Node (int data) {
            this → data = data;
            this → prev = NULL;
            this → next = NULL;
        }
};
```

Explanation –

```
Node * first = new Node (10);
Node * second = new Node (20);
Node * third = new Node (30);
first → next = second;
second → prev = first;
second → next = third;
third → prev = second;
```



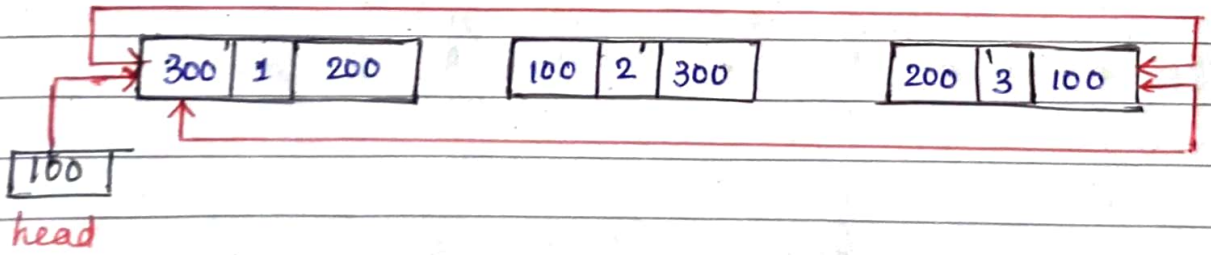

Now, we can move forward (10 → 20 → 30) or backward (30 → 20 → 10)

## 3. Circular linked list

- In a circular singly linked list, the last node points to the first node, instead of NULL, forming a loop.

Example -

```
class Node {
public :
        int data ;
        Node * next ;
        Node (int data) {
                this → data = data ;
                this → next = NULL ;
        }
};
Node * first  = new Node (10) ;
Node * second = new Node (20) ;
Node * third  = new Node (30) ;
first → next = second ;
second → next = third ;
third → next = first ; // last node connects back to first.
```



# Important (Must to Do)

Most famous and commonly Asked Linked list interview questions.

1) Reverse a linked list
2) Find the middle of the linked list
3) Detect a cycle in Linked list
4) Length of a linked list
5) Check if a linked list is a Palindrome.
6) Remove Duplicate from a sorted linked list.
7) Merge Two sorted linked list
8) Remove Nth Node from End of list
9) Intersection Point of Two Linked List
10) Cycle Length and Starting Node
11) Delete a Node without head pointer
12) Clone a linked list with random pointer
13) LRU Cache
14) Reverse Nodes in K-Group
15) Rotate linked list

SYNTAX ERROR | Abhishek Rathor

These handwritten notes are exclusively created for educational purposes under the SYNTAX ERROR brand. Unauthorized use, duplicated, or redistribution in any form is strictly prohibited.

for collaboration or queries : @ code.abhii07

# Operations of linked list

1. Traversal – Visiting each node in the list from start to end
   - Use to display or access all data.
   - Time : O(n)

2. Insertion – Adding a new node to the list.
   - → At beginning : Insert node before the head. O(1)
   - → At end : Add node after the last node. O(n)
   - → At position : Insert node at specific place. O(n)

3. Deletion – Removing a node from the list.
   - → From beginning : Remove the head : O(1)
   - → From end : Remove the last node. O(n)
   - → From position : Remove the node from a specific place. O(n)

4. Search – Find if a node with a specific value exists.

   Use to check or locate data.
   Time : O(n)

5. Update – Change the value of a node at a given position.
   Time : O(n)

6. Reverse – Reverses the direction of the list.
   Time : O(n)
   Space : O(1)

7. Length — counts the number of nodes in the list ... used to know the size.
Time : O(n).

@ SYNTAX ERROR
— Abhishek Pathak

# Tree

We read data structure, like an array, linked list, stack and queue in which all elements are arranged in a sequential manner.

A tree is a special type of data structure where data is stored in a hierachical (top to bottom) form.

It starts from root node.
Every node can have child node.
A node with no child is called a leaf.

## Basic Terminology

1. Root : Topmost node (starting point)
2. Parent : Node that has children
3. Child : Coming from parent
4. Leaf / External Node : Node with no children.
5. Internal Node : Node with at least one child.
6. Siblings : Nodes with the same parent
7. Degree : Number of children a node has.
8. Level : Distance from root (root is level 0)
9. Height / Depth : Largest path from root to leaf

10. Ancestors : Nodes above a given node in the tree like your parents, grand parents
The root is always an ancestor of all other nodes
11. Descendants : Nodes below a given node in the tree like your children and grandchildren etc.
All the nodes connected under your node are descendants.

Just Remember :
- Ancestors are above you
- Descendants are below you

## Types of Tree Data Structure :

1. Binary Tree
Each node can have at most 2 children (left and right)

2. Full Binary Tree
Every node has 0 or 2 children only

3. Perfect Binary Tree
All internal nodes have 2 children, and all leaf nodes are at the same level.

4. Complete Binary Tree
All levels are completely filled except maybe the last, which is filled left to right

classmate
Date
Page

5. Skewed Binary Tree
Node only have one child
- Left skewed : Nodes have only left child
- Right skewed : Nodes have only right child.

① Binary Tree (BT) — Each node has at most two children — left and right

Basic Node Structure

```
class Node {
public :
    int data;
    Node* left; // pointer to left child
    Node* right; // pointer to right child
    Node (int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};
```

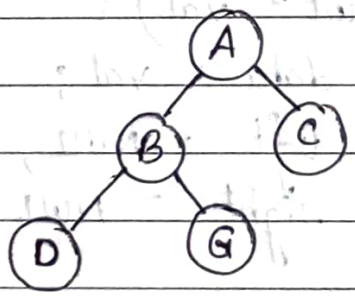
Properties of Binary Tree —

- At each level of i, the maximum number of nodes is 2.
- The height of tree is longest path from root node to leaf node. In general, maximum number of nodes possible at height is $(2^0 + 2^1 + 2^2 + 2^3 + \ldots 2^n)$

- The minimum number of nodes possible at height is equal to h+1.
- If number of nodes is minimum, then height of tree would be maximum.
- Minimum height can be computed as:
  $h = \log_2 (n+1) - 1$
- Maximum height can be computed as:
  $h = n - 1$

② Full / Proper / Strict Binary Tree

Every node has 0 or 2 children




Properties –

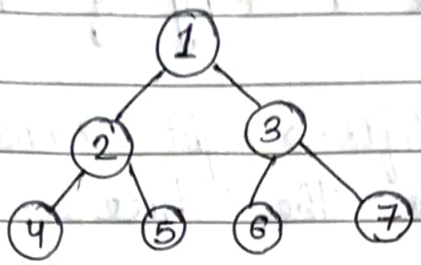
Maximum number of nodes : $2^{h+1} - 1$

Minimum number of nodes : $2 * h - 1$

Minimum height $\log_2 (n+1) - 1$

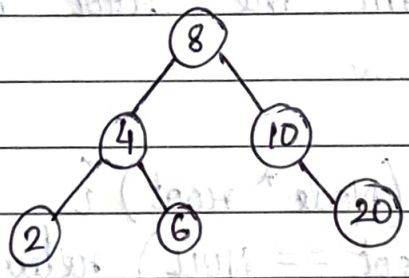
Maximum height $h = \frac{n+1}{2}$

③ Perfect Binary Tree




Note: All the perfect Binary Trees are complete binary trees as well as the full Binary tree as But, vice versa is not true, all complete binary trees are full binary trees are the perfect Binary Trees.

Binary Search Tree (BST) –




- Key Operations –
- Insert: Recursively add elements in correct order
- Search: Stand at root, compare, move left or right.
- Delete: Handle 0, 1 or 2 children cases.

Tree traversals: ways to visit all nodes in a tree:

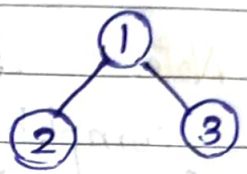
- In order (left → Root → Right)

Prints nodes in increasing order in BST

- Preorder (Root → Left → Right)
  Used to create a copy of the tree

- Post order (Left → Right → Root)
  used to delete the tree.

- Inorder : 2 1 3
- Preorder : 1 2 3
- Postorder : 2 3 1




④ Height of Tree

Def → The number of edges on the longest path from the root to a leaf.

Code :

```
int height (Node * root) {
    if (root == NULL) return 0;
    int leftHeight = height (root → left);
    int rightHeight = height (root → right);
    return max (leftHeight, rightHeight) + 1;
}
```

⑤ Diameter of Tree – The longest path between any two nodes (not necessarily through the root)

Formula –

diameter = max (leftHeight + rightHeight + 1)

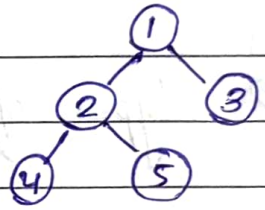
classmate
Date
Page

## ⑥ Views of Binary Trees

- Top view – Nodes visible from the top when tree is viewed horizontally.
- Bottom view – Nodes visible from the bottom.
- Left view – Leftmost node at every level.
- Right view – Rightmost node at every level.

## ⑦ Zigzag Traversal : Level order but alternate levels are reversed.

Zigzag – 1, 3, 2, 4, 5.




## Top 10 Tree Interview Questions

1. Inorder / Preorder / Postorder
2. Check if a Binary Tree is a BST
3. Lowest Common Ancestor (LCA)
4. Height / Depth of a tree
5. Diameter of a Binary tree
6. Level order Traversal (BFS)
7. Invert a Binary Tree (Mirror Tree)
8. Serialize and Deserialize a Binary Tree
9. Path Sum
10. Convert sorted array to Balanced BST

# Heap & Priority Queue

⇒ Def → A heap is a special complete Binary tree that satisfies the heap property.

- Min Heap : Parent node ≤ child Node.
- Max Heap : Parent node ≥ child Node.

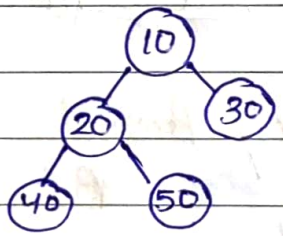
⇒ Properties :

- Always complete (all levels filled except posibly the last)
- Stored in a array (index-based logic)

⇒ Types of Heaps :

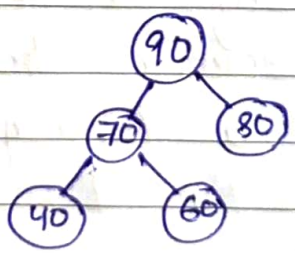
- Min Heap – Root contains the minimum element.

Example




- Max Heap – Root contains the maximum element.

Example




⇒ Heap Operations

| Operations | Time Complexity |
|---|---|
| Insert | O (log N) |
| Delete (root) | O (log N) |
| Peek (min/max) | O(1) |

# C++ Priority Queue

C++ provides priority-queue is <queue> header.

- Default : Max-Heap based on natural ordering

Create Min-Heap

```
priority_queue<int, vector<int>
greater<int> minHeap;
```

Create Max-Heap

```
priority_queue<int> maxHeap; // by default max-heap
```

Operations

```
minHeap.push(10);          // Insert
cout << minHeap.top();     // Minimum element
minHeap.pop();             // Remove minimum
```

Heapify (Build Heap)

→ converts an array into a heap in O(n) time.
→ ensures heap property maintained from bottom-up.

Applications

- used in Heap Sort
- creating priority queues.

## Heap Sort

Steps :

1. Convert array to max-heap
2. Swap root (max) with last element
3. Reduce heap size and re-heapify
4. Repeat

Time Complexity : $O(N \log N)$
Space Complexity : $O(1)$ (in place)

## K' Largest / Smallest Element

- K largest
  → Use Min-Heap of size k
  → Keep top k largest seen so far

- K Smallest
  → Use Max-Heap of size k.

Time Complexity : $O(N \log N)$

## Median in a Stream

Problem : Given a stream of integers, return the median after every new element

Approach
Use 2 heaps :

- Max-Heap → for lower half
- Min-Heap → for upper half

```
priority_queue<int> maxHeap; // lower half
priority_queue<int, vector<int>, greater<int>>
                    minHeap; // upper half
```

- Balance both heaps after every insertion
- Median:
  - If count is even → average of tops
  - If count is odd → top of larger heap

Example:

Input: [5, 15, 1, 3]

Output: 5, 10, 5, 4

## Summary Table

| Topic | Heap Used | Time Complexity |
|---|---|---|
| Insert / Delete | Min or max Heap | O(log N) |
| Heapify | Min or max Heap | O(N) |
| Heap Sort | Max Heap | O(N log N) |
| K largest / Smallest | Min or max Heap | O(N log K) |
| Median is Stream | 2 Heaps | O(log N) per insert |

# Greedy Algorithms

→ A greedy algorithm is an approach builds up a solution piece by piece, always choosing the locally optimal option at each step with the hope of finding a global optimum.

⇒ Key Characteristics :

- Make decisions greedily at every step.
- Does not revisit earlier decisions.
- works only for problems with greedy property and optimal substructure.

⇒ General Approach for Greedy Problems :

1. Sort data (e.g, by end time, profit, ratio)
2. Apply a greedy choice strategy
3. Make decisions step-by-step

## ① Activity Selection Problem

Problem : Given N activities with start and end times, select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time.

# Greedy Strategy

→ Always pick the activity that ends earliest
→ Time Complexity : O(N log N) (due to sorting)

Example

```
#include <bits/stdc++.h>
using namespace std;
class Activity {
    public:
        int start, end;
        Activity(int s, int e) {
            start = s;
            end = e;
        }
};
class ActivitySelection {
public:
    static void solve() {
        vector<Activity> activities = {
            Activity(1,3),
            Activity(2,4),
            Activity(3,5),
            Activity(6,7),
            Activity(8,9),
            Activity(0,6)
        };
        //Sort activities by end time
        sort(activities.begin(), activities.end(),
[](Activity &a, Activity &b) {
```

```
        return a.end < b.end;
    });
    int count = 1;
    int lastEnd = activities[0].end;
    for (int i=1; i< activities.size(); i++){
        if (activities[i].start >= lastEnd) {
            count++;
            lastEnd = activities[i].end;
        }
    }
    cout << "Max activities : " << cout
                                << endl;
  }
};
int main() {
    ActivitySelection :: solve();
    return 0;
}
```

## (2) Huffman Coding

Problem: Given characters and their frequencies build an optimal prefix code (no code is prefix of another) with minimum average length.

⇒ Greedy Strategy:

- Use a Min Heap to combine lowest frequency nodes.

- Build a binary tree and assign codes from top to bottom.

⇒ Time Complexity : $O(N \log N)$

⇒ Applications :
- File compression (ZIP, JPEG)
- Data encoding

## (3) Job Scheduling Problem.

Problem : Given N jobs with deadlines and profits, schedule jobs to maximize total profit such that only one job can be scheduled at a time and within its deadline.

⇒ Greedy Strategy :
- Sort jobs by profit in descending order.
- Schedule each job in the latest possible available slot before its deadline.

⇒ Time Complexity : $O(N \log N + N^2)$

⇒ C++ Skeleton :

```
// Structure for a Job
class Job {
    public :
        int deadline, profit;
        Job (int d, int p) {
            deadline = d;
            profit = p;
    };      }
```

# Questions Must to Do....

1. Fractional Knap Sack
2. Minimum Platforms Problems.

## Tries

→ A trie (pronounced as "try") is a tree-based data structure used for storing strings in an effecient way especially when the dataset contains many strings with common prefixes.

⇒ Key Features :

- Every node represents a character
- Paths from root to leaf represent words
- used for prefix-based problems.

⇒ Applications of Trie :

- Dictionary word storage
- Autocomplete
- Spell checker
- Prefix Search
- word segmentation (word Break Problem)

→ Trie Node Structure in C++

```
class TrieNode {
public:
    TrieNode* children[26];   // For 26 lowercase
                              English letters
    bool isEndOfWord;
    TrieNode() {
        isEndOfword = false;
        for (int i = 0; i < 26; i++) {
            children[i] = NULL;
        }
    }
};
```

1. Insert a word into Trie

→ Idea : For each character in the word :
- Move to the corresponding child node
- Create a node if it doesn't exist.

→ Time Complexity : O(L)
where L = length of word.

→ C++ Code :

```
void insert(TrieNode* root, string word)
{
    TrieNode* node = root;
    for (char ch : word) {
        int index = ch - 'a';
        if (node->children[index] == NULL) {
        node->children[index] = new TrieNode();
    }
```

```
        node = node -> children[index];
    }
    node -> isEndOfWorld = true;
}
```

2. Search a Word in Trie

→ Idea:

- Traverse the Trie following each character in the word.
- If a required node is missing → return false

→ C++ Code:

```
bool Search(TrieNode* root, string word)
{
    TrieNode* node = root;
    for (char ch : word) {
        int index = ch - 'a';
        if (node -> children[index] == NULL) {
            return false;
        }
        node = node -> children[index];
    }
    return node -> isEndOfWord;
}
```

3. Prefix Check (starts With)

→ Idea :

• Same as search, but don't check for isEndOfWorld.

→ Time Complexity : O(L) where L = length of prefix.

→ C++ Code :

```
bool startsWith (TrieNode* root, string prefix)
{
  TrieNode* node = root.
  for (char ch : prefix) {
      int index = ch - 'a';
      if (node → children [index] == NULL) {
          return false;
      }
      node = node → children [index];
  }
  return true;
}
```

@ SYNTAX ERROR
— Abhishek Rathor

4. Word Break Problem

Problem : Given a string s and a dictionary of words, determine if s can be segmented into a space-separated sequence of dictionary words.

→ Approach :

Use Trie + Recursion + Memoization

→ Time Complexity :
$O(N \times L)$ where $N$ = length of string
$L$ = length of longest word

5. Longest word with All Prefixes

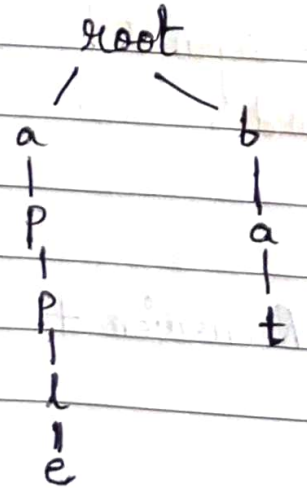
→ Problem : Find the longest word in the dictionary such that all prefixes of the word are also in the dictionary.

→ Example :
Input : ['W', 'Wo', 'Wor', 'Worl', 'World']
Output : "World"

→ Idea : Use DFS on Trie and only go to child nodes that are marked as isEndOfWord = true

Trie Tree Visual Example

Let's say we insert : "apple", "app", "bat"



# Summary Table

| Operation | Time Complexity | Notes |
|---|---|---|
| • Insert | O(L) | L = length of word |
| • Search | O(L) | Fast lookup |
| • Prefix | O(L) | Useful for autocomplete |
| • word Break | O(N × L) | Use Trie + DP |
| • Longest word | O(N × L) | DFS + all prefixes exist check |

# Example : Fibonacci Number

1. Memorization (Top-Down)

```
#include <bits/stdc++.h>
using namespace std;
int fibMemo (int n, vector<int>&dp) {
        if (n <= 1) return n;
        if (dp[n] != -1) return dp[n];
        return dp[n] = fibMemo(n-1, dp) +
fibMemo (n-2, dp);
}
```

2. Tabulation (Bottom-Up)

```
int fibTab (int n) {
        vector<int> dp (n+1);
        dp[0] = 0;
```

```
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

# 0/1 Knapsack Problem

→ Given: weights and values of n items, max weight

→ Goal: Maximise value with 0 or 1 copy of each item

→ DP Formula:

```
if (wt[i] <= W)
    dp[i][w] = max(dp[i-1][w], val[i] + dp[i-1]
                                    [w - wt[i]]);
```

→ Time Complexity: $O(n \cdot W)$

→ Space Complexity: $O(n \cdot W)$

# Unbounded Knapsack

You can take unlimited instances of an item

→ Formula:

```
if [wt[i] <= W)
    dp[i][w]
```

# Subset Sum Problem

Check: Is there a subset with sum = k?

Boolean DP :

```
dp[i][j] = dp[i-1][j] || dp[i-1][j-arr[i]];
```

→ Partition Equal Subset Sum

Can we partition array into two subsets with equal sum ?

Reduce to subset sum with sum/2.

→ Longest Common Subsequence (LCS)

Given : Two strings s1, s2
Goal : Length of longest common subsequence

Formula :

```
if (s1[i-1] = s2[j-1])
    dp[i][j] = 1 + dp[i-1][j-1];
else
    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

Optimized O(n log n) using binary Search

→ Matrix Chain Multiplication (MCM)

Given : Matrix dimensions [40, 20, 30, 10, 30]
Goal : Minimum cost to multiply them

Recursion with DP :

```
for k = i to j-1;
    cost = dp[i][k] + dp[k+1][j] + arr[i-1] * arr[k] * arr[j];
```

⇒ DP on Grids

Goal : Find paths, min/max sum from (0,0) to (n-1, m-1)

Formula :

$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$

⇒ DP on Trees

Used In : Tree diameter, node values, etc.

Recursive DP with postorder traversal

⇒ Digit DP (Advanced)

Used For : Count numbers with certain properties within a range.

Parameters :

- pos : index in digit
- tight : is prefix same as original?
- Sum : Current sum or count

# Summary Table

| Problem | Time Complexity | DP Type |
|---|---|---|
| 0/1 Knapsack | $O(N * W)$ | 2D Table |
| Subset Sum | $O(n * Sum)$ | Boolean DP |
| LCS | $O(n * m)$ | 2D Table |
| LIS | $O(n \log n)$ | Optimized |
| MCM | $O(n^3)$ | DP Partition |
| DP on Grid | $O(n * m)$ | 2D Grid |

# Graphs

A graph is a non-linear data structure consisting of
- vertices (nodes)
- edges (connections)

## # Types

- Directed or Undirected
- Weighted or Unweighted
- Cyclic or Acyclic
- Connected or Disconnected

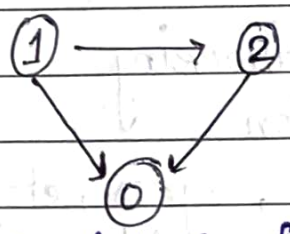
## Graph Representation

1. Adjacency List
- An array / list where each index holds a list of neighbours.
- Space Efficient : O(V+E)

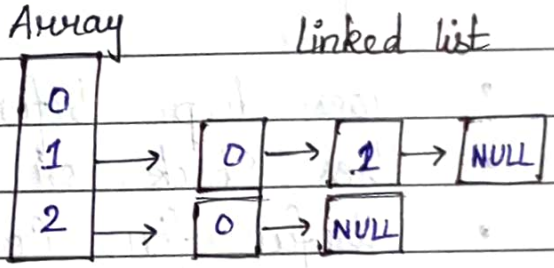
Example

```
int main () {
    int V = 3; // no. of vertices
    vector<vector<int>> adj(V);
    //edge 0 → 1
    adj[0].push_back(1);
    //edge 1 → 2
    adj[1].push_back(2);
```

```
//edge 2 → 0
adj[2].push_back(0);
// printing adjacency list
for (int i=0; i<V; i++) {
    cout << i << " -> ";
    for (int v : adj[i]) {
        cout << v << " ";
    }
    cout << endl;
}
return 0;
}
```




Directed Graph




Adjacency list

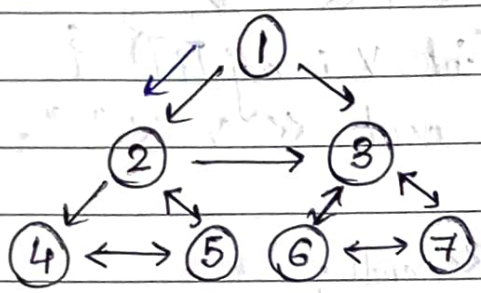
2. Adjacency Matrix :

- A 2D array graph [V][V]
- graph [i][j] = 1 if edge from i to j exists
- Space : $O(V^2)$

# Breadth - First Search (BFS)

- level by - level traversal
- Uses Queue
- Good for finding shortest path in unweighted graph.

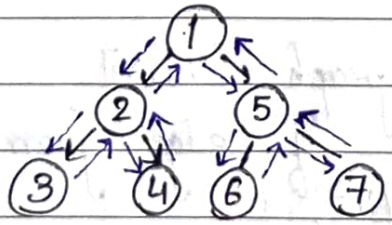
- Time Complexity : $O(V+E)$
- Traverse through one level of children nodes, then traverse through the level of grand children nodes (and so on...)




Output : 1, 2, 3, 4, 5, 6, 7

# Depth - First Search (DFS)

- Goes deep before backtracking
- Uses Stack or Recursion
- Useful for Components, cycles, etc.
- Traverse through left subtree (s) first, then traverse through the right subtree (s)




- Output : 1, 2, 3, 4, 5, 6, 7
- Time Complexity : $O(V+E)$

# Cycle Detection

1. In Undirected Graph
   - Use DFS + parent tracking
   - If a visited node is reached and not the parent → cycle.

2. In Directed Graph
   - DFS + recursion stack
   - If a node is revisited in recursion → cycle

# Dijkstra's Algorithm (Shortest Path)

- used for : Positive weight edges only
- Steps : Use Min-Heap (Priority Queue)
  Track shortest distance from source
- Time Complexity : $O((V+E) \log V)$

# Bellman – Ford Algorithm

- used for :
  Graphs with negative weights.
  can detect negative cycles.
- Steps :
  Relax all edges $V-1$ times
  One more pass to check for negative cycles
- Time : $O(V \times E)$

# Topological Sort
used for directed acyclic graphs (DAG)

Kahn's Algorithm (BFS - based)
- count indegrees
- use Queue for nodes with 0 indegree
- Process & reduce in degree of neighbours
- Time : O(V+E)

# Disjoint Set (Union - Find)
- used in :
  - cycle detection
  - Kruskal's MST
- Operations :
  Find(x) : Find parent
  Union (x,y) : Combine sets

Optimization :
- Path Compression
- Union by Rank

# Minimum Spanning Tree (MST)
Spanning Tree → covers all nodes with minimum total edge weight
Kruskal's Algorithm :
- Greedy + Union Find
- Sort edges by weight
- Add edge if it doesn't create a cycle.

Time : $O(E \log E)$

Prim's Algorithm

- use Min-Heap (priority Queue)
- start from any node, add shortest edge to MST Times : $O((V+E) \log V)$

# Summary Table

| Topic | Time Complexity | Notes |
|---|---|---|
| BFS/DFS | $O(V+E)$ | BFS uses queue, DFS uses stack |
| Dijkstra | $O((V+E) \log V)$ | No negative weights |
| Bellman-Ford | $O(V \times E)$ | can handle -ve weights |
| Kahn's Topo Sort | $O(V+E)$ | BFS-based, For DAG |
| Union Find (Disjoint Set) | $O(\alpha(N))$ | For Kruskal, cycle detection |
| Kruskal's MST | $O(E \log E)$ | greedy using sorted edges |
| Prim's MST | $O((V+E) \log V)$ | greedy using priority queue |

# Sliding Window & Two Pointers

## # What is Sliding Window?

Sliding Window is a technique for optimizing problems involving arrays or strings by using a moving window (subarray / substring) of elements.

Types of Windows

1. Fixed-size windows: window of fixed size k slides across
2. Variable-size window: Size of window changes based on conditions (like distinct chars, sum)

## # What is Two Pointers?

Two Pointer is a technique where we use two indices (pointers) to traverse a sequence to find a condition or pattern (e.g., sorted array, substring)

1. Fixed-size Window - Maximum Sum of Subarray of size k

⇒ Problem: Find the maximum su of subarray of size k.

@ SYNTAX ERROR
– Abhishek Rathor

⇒ C++ Code:

```
int maxSumSubarray (vector<int>& arr, int k) {
    int maxSum = 0, windowSum = 0;
    // First window sum
    for (int i = 0; i<k; i++) {
        windowSum += arr[i];
    }
    maxSum = windowSum;
    for (int i=k; i<arr.size(); i++)
    {
        windowSum += arr[i] - arr[i-k]
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}
```

2. Variable - size Window - Longest Substring without Repeating characters.

⇒ Problem: Given a string, find the length of the longest substring without repeating characters

⇒ C++ Code:

```
int length (string s) {
    unordered_set<char> set;
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length();
                right++) {
        while (set.count(s[right])) {
            set.erase(s[left++]);
    }
```

```
        set.insert(s[right]);
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

3. Maximum in Sliding Window (Deque Approach)

→ Problem: Find maximum at each subarray of size k.

⇒ C++ Code:

```
vector<int> maxSlidingWindow(vector<int>& nums,
                             int k) {
    deque<int> dq; // stores indices
    vector<int> result;
    for (int i=0; i< nums.size(); i++) {
        if (!dq.empty() && dq.front() ==
                i-k) {
            dq.pop_front();
        }
        while (!dq.empty() && nums[dq.back()]
                < nums[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        if (i>=k-1) {
            result.push_back(nums[dq.front()]);
        }
    }
    return result;
}
```

4. Trapping Rainwater Problem

⇒ Problem: Given height of bars, calculate how much water it can trap after raining.

⇒ C++ Code:

```
int trap(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax)
            {
                leftMax = height[left];
            } else {
                water += leftMax - height[left];
            }
            left++;
        } else {
            if (height[right] >= rightMax){
                rightMax = height[right];
            } else {
                water += rightMax - height
                         [right];
            }
            right--;
        }
    }
    return water;
}
```

⇒ Time : O(n) Space : O(1)

⇒ Example for Trapping Rain water :
Input : height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output : 6

# Summary Table

| | Problem | Approach | Time | Space |
|---|---|---|---|---|
| 1. | Max Sum Subarray (size k) | Fixed window | O(n) | O(1) |
| 2. | Longest Substring without Repeat | Two pointer/ set | O(n) | O(n) |
| 3. | Max in Sliding Window | Deque | O(n) | O(k) |
| 4. | Trapping Rainwater | Two pointers | O(n) | O(1) |

SYNTAX ERROR | Abhishek Rathor

These handwritten notes are exclusively created for education purposes under the SYNTAX ERROR brand.
Unauthorized use, duplicated or redistribution in any form is strictly prohibited.
for collaboration or queries :
@ code.abhii07

# Bit Manipulation

Bit Manipulation is a technique of performing operations on data at the bit level. It is highly effecient and often used for optimization in problems involving numbers, sets, toggles etc.

## # Common Bitwise Operators

| Operator | Symbol | Example (a=5, b=3) | Result (in bits) |
|---|---|---|---|
| AND | & | a&b = 1 | 0101 & 0011 = 0001 |
| OR | \| | a\|b = 7 | 0101 \| 0011 = 0111 |
| XOR | ^ | a^b = 6 | 0101 ^ 0011 = 0110 |
| NOT | ~ | ~a = -6 | ~0101 = 1010 (in 2's comp.) |
| Left Shift | << | a<<1 = 10 | 0101 << 1 = 1010 |
| Right Shift | >> | a>>1 = 2 | 0101 >> 1 = 0010 |

**Note** Bit indexing starts from right to left, starting at 0.

### 1. Check if a Number is a Power of Two

A number is a power of 2 if it has only one set bit (eg., $2^1 = 10$, $4 = 100$).

Code :

```
bool isPowerOfTwo(int n) {
    return n>0 && (n & (n-1)) == 0;
}
```

Explanation :

- For power of two : only one bit is 1.
- n & (n-1) removes the lowest set bit.
- So result should be 0 for power of 2.

2. Count Set Bits (No. of 1s in Binary)

Problem : Count how many sets bits (1s) are in the binary representation of a number.

Code :

```
int countSetBits(int n) {
    int count = 0;
    while (n>0) {
        count += (n & 1); // check last bit
        n >>= 1;  // right shift
    }
    return count;
}
```

Time Complexity : $O(\log n)$

3. Find the Single Number (All others appear twice)

Problem : Give an array where every element appears twice except one, find that one

Code:

```
int singleNumber(vector<int>& nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num; // XOR cancels out duplicates
    }
    return result;
}
```

Why it works?

- $a \wedge a = 0$
- $a \wedge b = b$

## 4. XOR of All Number from 1 to n

There is a pattern:

- $n \% 4 == 0 \rightarrow n$
- $n \% 4 == 1 \rightarrow 1$
- $n \% 4 == 2 \rightarrow n+1$
- $n \% 4 == 3 \rightarrow 0$

Code:

```
int XorUptoN(int n) {
    if (n % 4 == 0) return n;
    if (n % 4 == 1) return 1;
    if (n % 4 == 2) return n+1;
    return 0;
}
```

5. Find Missing Number (0 to n)

Problem: Given array of size n with elements from 0 to n. Find the missing number.

Code:

```
int missingNumber (vector<int>& nums) {
    int xorVal = 0;
    for (int i = 0; i < nums.size(); i++) {
        xorVal ^= i ^ nums[i];
    }
    return xorVal ^ nums.size();
}
```

6. Get i-th Bit of a Number

Code:

```
int getIthBit (int n, int i) {
    return (n >> i) & 1;
}
```

7. Set i-th Bit

Code:

```
int setIthBit (int n, int i) {
    return n | (1 << i);
}
```

8. Clear i-th Bit

Code:

```
int clearIthBit (int n, int i) {
    return n & ~(1 << i);
}
```

Examples :

Example 1 : is Power of Two (8)
Binary : 1000
8 & 7 = 0 → True

Example 2 : count Bits (13)
Binary : 1101 → 3 ones

Example 3 : single Number ([2, 2, 1])
2 ^ 2 ^ 1 = 0 ^ 1 = 1

# Summary Table

| Problem Type | Time | Space |
|---|---|---|
| Check Power of Two | O(1) | O(1) |
| Count Set Bits | O(log n) | O(1) |
| Find Single Number | O(n) | O(1) |
| XOR of 1 to n | O(1) | O(1) |
| Missing No. (0 to n) | O(n) | O(1) |
| Get i-th Bit | O(1) | O(1) |
| Set i-th Bit | O(1) | O(1) |
| Clear i-th Bit | O(1) | O(1) |

# Segment Tree & Fenwick Tree

## What is Segment Tree?

A segment Tree is a binary tree structure used to perform effecient range queries and updates (eg. sum, min, max) in logarithmic time.

Useful for:

- Range sum queries
- Range min / max queries
- Updating values effeciently

Basic Concept:

- Each node in the segment Tree represents a segment (range) of the input array.
- Leaf nodes store values of the array.
- Internal nodes store results of queries (sum/min/max) on child segments.

Time & Space Complexities

| Operation | Time |
|---|---|
| Build | O(n) |
| Query (Range) | O(log n) |
| Update (Point) | O(log n) |
| Space | O(4n) |

# Example Problem : Range Sum Query

Problem Statement : Given an array = {3, 5, 7, 9, 11}. build a segment Tree to find the sum in any given range efficiently.

C++ Code :

```
class SegmentTree {
    vector<int> tree;
    int n;
public:
    SegmentTree(vector<int>& arr) {
        n = arr.size();
        tree.resize(4*n);
        build(arr, 0, n-1, 1);
    }
    void build(vector<int>& arr, int l, int r,
    int i) {
        if (l == r) {
            tree[i] = arr[l];
            return;
        }
        int mid = (l+r)/2;
        build(arr, l, mid, 2*i);
        build(arr, mid+1, r, 2*i+1);
        tree[i] = tree[2*i] + tree[2*i+1];
    }
    int query(int ql, int qr) {
        return queryUtil(0, n-1, ql, qr, 1);
    }
}
```

```
int queryUtil (int l, int r, int ql, int qr,
                int i) {
    if (qr < l || ql > r) return 0;
// No overlap
    if (ql <= l && r <= qr) return tree[i];
    int mid = (l+r)/2;
    return queryUtil (l, mid, ql, qr, 2*i)+
        queryUtil (mid+1, r, ql, qr, 2*i+1);
}
void update (int idx, int val) {
    updateUtil (0, n-1, idx, val, 1);
}
void updateUtil (int l, int r, int idx, int val,
    int i) {
    if (l == r) {
        tree[i] = val;
        return;
    }
    int mid = (l+r)/2;
    if (idx <= mid) updateUtil (l, mid, idx,
        val, 2*i);
    else updateUtil (mid+1, r, idx, val, 2*i+1);
    tree[i] = tree[2*i] + tree[2*i+1];
}
};
```

## Explanation

- Build Tree : O(n)
- Query Range Sum : O(log n)
- Update Value : O(log n)

## Example

- arr = {3, 5, 7, 9, 11}
- Query (1, 3) → 5 + 7 + 9 = 21
- Update (1, 10) → arr becomes {3, 10, 7, 9, 11}
- Query (1, 3) → 10 + 7 + 9 = 26.

## Lazy Propagation

used to perform range updates efficiently in segment Trees.

⇒ Problem : You want to add a value v to all elements in a range [L, R]. Naive update is O(n), but lazy propagation does it in O(log n).

⇒ How it works :

- use a lazy[] array to store delayed updates
- propagate updates when visiting nodes (only when necessary)

→ Time : O(log(n)) for both query and update

## Fenwick Tree (Binary Indexed Tree - BIT)

A compact and efficient data structure for range queries and point updates

→ Use Case :
- Prefix sum queries
- Point updates

→ Core Idea :
Each index stores sum of previous elements, cleverly designed with binary operations.

→ Time Complexity :

| Operation | Time |
|---|---|
| Update | O(log n) |
| Prefix Query | O(log n) |
| Space | O(log n) |

→ C++ Code :

```
class FenwickTree {
    vector<int> bit;
    int n;
public:
    FenwickTree(int size) {
```

```
        n = size;
        bit.assign(n+1, 0);
    }
    void update(int idx, int delta) {
        while (idx <= n) {
            bit[idx] += delta;
            idx += idx & -idx;
        }
    }
    int query(int idx) {
        int sum = 0;
        while (idx > 0) {
            sum += bit[idx];
            idx -= idx & -idx;
        }
        return sum;
    }
    int rangeQuery(int l, int r) {
        return query(r) - query(l-1);
    }
};
```

⇒ Example: arr = [0, 1, 3, 4, 8, 6, 1, 4, 2] (1-based)

- update (3, +2) ⇒ add 2 to index 3
- query (5) → sum of 5 Elements

@ SYNTAX ERROR
— Abhishek Rathor

# Summary Table —

| Structure | Range Query | Point update | Space | Best For |
|---|---|---|---|---|
| Segment Tree | O(log n) | O(log n) | O(4n) | Range min/max/sum |
| Lazy segment | O(log n) | O(log n) | O(4n) | Range update + range queries |
| Fenwick Tree | O(log n) | O(log n) | O(4n) | Prefix sum & point update |

## Interview & Contest Specific Topics

# C++ Standard Template Library (STL)

The C++ STL is a powerful library that provides ready-to-use classes and functions for storing and manipulation data.
It contains containers, iterators, algorithms, and function objects.

### Common Containers

| Container Type | Description |
|---|---|
| Sequence Containers | store data in linear order. Examples: vector, list, deque |

| | |
|---|---|
| Associative Containers | store unique elements with fast access using keys. Examples: set, map, multiset |
| Unordered Containers | Based on hashing, allow average O(1) lookup. Examples: unordered_set, unordered_map. |
| Container Adaptors | Provide restricted interface for specific usage. Examples: stack, queue, priority_queue. |

## Common Implementations

| Type | Implementation | Properties |
|---|---|---|
| Vector | Dynamic array | Random access, automatic resizing |
| List | Doubly linked list | Fast insertion/deletion anywhere |
| Deque | Double-ended queue | Insertion/deletion at both ends. |
| Set | set (ordered, balanced BST) | stores unique elements in sorted order |
| Unordered Set | unordered_set (hashing) | stores unique elements, no order, O(1) average lookup |

| | |
|---|---|
| Associative Containers | store unique elements with fast access using keys. Examples: set, map, multiset |
| Unordered Containers | Based on hashing, allow average O(1) lookup. Examples: unordered_set, unordered_map. |
| Container Adaptors | Provide restricted interface for specific usage. Examples: stack, queue, priority_queue. |

## Common Implementations

| Type | Implementation | Properties |
|---|---|---|
| Vector | Dynamic array | Random access, automatic resizing |
| List | Doubly linked list | Fast insertion/deletion anywhere |
| Deque | Double-ended queue | Insertion/deletion at both ends. |
| Set | set (ordered, balanced BST) | stores unique elements in sorted order |
| Unordered Set | unordered_set (hashing) | stores unique elements, no order, O(1) average lookup |

| | | |
|---|---|---|
| Map | map (ordered, balanced BST) | stores key-value pairs sorted by key |
| Unordered Map | unordered_map (hashing) | stores key-value pairs, O(1) average lookup |
| Queue | queue | FIFO (First-In-First-Out) |
| Priority Queue | priority_queue | Max-heap/Min-heap behaviour |
| Stack | stack | LIFO |

⇒ Example: Using unordered_map

```
int main (){
    unordered_map <string, int > mp;
    mp ['apple'] = 2;
    mp ['banana'] = 4;
    cout << 'Apple count = ' << mp ['apple']<<
                                endl; // Output: 2

    return 0;
}
```

⇒ Time and Space Complexity Analysis

Why It Matters:

- Essential for evaluating performance of code
- crucial in interviews to justify your approach.

Common Complexities

| Complexity | Meaning |
|---|---|
| $O(1)$ | constant time (e.g., hash lookup in unordered_map) |
| $O(\log n)$ | Logarithmic time (e.g., search in set/map) |
| $O(n)$ | Linear time (e.g., traversing vector, list) |
| $O(n \log n)$ | Sorting algorithms (Merge Sort, Quick Sort) |
| $O(n^2)$ | Nested loops (e.g., comparing all pairs) |

# Binary Search on Answer

- Apply binary search on the search space, not just the array.
- Example: Minimum speed to deliver packages in k hours.

# Prefix Sum / Difference Array

- used to pre-compute data for fast range queries.
- common in range sum, range update problems.

# Hashing

- use unordered_map / unordered_set in C++ STL.
- Applications : frequency counting, uniqueness checks, etc.

# Common Mistakes & Debugging Tricks

X Common Mistakes & ✓ Fixes

| Mistake | Fix/Tip |
|---|---|
| Off-by-one errors | check loop boundaries carefully |
| Infinite loops | verify exit conditions |
| using == with strings | use string1 == string2 in C++ (works correctly) |
| Not handling edge cases | always test with empty / single-element inputs. |
| Wrong input/output formula | read the problem carefully |

# Debugging Tricks

- use cout statements for step-by-step tracing.
- Example :

```
cout << "i = " << i << ", value = "
     << arr[i] << endl;
```

# Testing & Edge Cases

- Test edge cases: empty arrays, minimum constraints
- Run code with small input first, before full submission
- Use custom test cases (if allowed in the IDE/ contest platform)

# Summary Table

| Topic | Key Use Case |
|---|---|
| C++ STL Containers | fast data storage & access |
| Time/Space Complexity | analyze and optimize solutions |
| Sliding Window/ 2 Pointers | Efficient array & string problems. |
| Debugging & Testing | Fix logic, performance, and edge issues. |

SYNTAX ERROR | Abhishek Error
These handwritten notes are exclusively created for education purposes under the SYNTAX ERROR brand.
Unauthorized use, duplicated or redistribution in any form is strictly prohibited.
for collaboration or queries : @ code.abhii07